Factorización de Cholesky

Memoria experimental ordenada

De la referencia secuencial y la validación a la comparación final entre MPI y LAPACK/OpenBLAS.

Mapa de lectura

PARTE I — **Fundamentos, validación y decisiones locales** — 3–10

Primero se fija qué se calcula y cómo se comprueba; después se comparan las variantes escalares.

PARTE II — **Diseño MPI y primera implementación distribuida** — 11–25

Las decisiones sobre propiedad de columnas aparecen inmediatamente antes de la campaña que las pone a prueba.

PARTE III — **Cruce de rendimiento y comparación de algoritmos MPI** — 26–34

El modelo de costos se presenta cuando ya existe un candidato MPI y justo antes de medir cuándo compensa.

PARTE IV — **Flujo de datos y cierre local** — 35–50

La investigación pasa del kernel al recorrido completo: generación, reunión, distribución y programa candidato.

PARTE V — **Campañas en el clúster y comparación final** — 51–77

La evidencia local se contrasta con el entorno real, el cruce formal, la escalabilidad y LAPACK.

PARTE I

I

Fundamentos, validación y decisiones locales

¿La implementación es correcta y qué organización local conviene antes de introducir MPI?

Cholesky: el problema que vamos a resolver

Queremos factorizar una matriz real A, simétrica y definida positiva, como el producto de una matriz triangular inferior L y su transpuesta:

$$A = LL^T$$

Por encima de la diagonal de L todos los elementos son cero. Elegimos su diagonal positiva; con esa condición el factor es único. Esta factorización permite resolver Ax = b mediante los sistemas triangulares Ly = b y $L^T x = y$.

Condiciones de entrada

$$A = A^T, \qquad x^T A x > 0 \quad \text{para todo } x \neq 0$$

La primera condición es simetría; la segunda es definición positiva. En aritmética exacta garantizan que los pivotes de Cholesky sean positivos, sin necesidad de intercambiar filas. El código comprueba los pivotes para detectar entradas o resultados inválidos.

De dónde salen las dependencias

Al desarrollar el producto matricial, con índices desde cero, obtenemos:

$$a_{ij} = \sum_{k=0}^{\min(i,j)} l_{ik} l_{jk}$$

Una columna nueva utiliza contribuciones de las anteriores. Left-looking y right-looking cambian cuándo se aplican esas contribuciones, no el problema matemático. Ambas variantes escalares realizan aproximadamente $n^3/3$ operaciones de punto flotante.

Las tres decisiones del trabajo

Primera: qué variante conviene estudiar y qué muestran sus tiempos.

Segunda: cuándo el ahorro de cálculo justifica usar más procesos.

Tercera: cómo repartir los datos para mantener trabajo útil en esos procesos.

Todo el cálculo, la generación de matrices y la futura implementación MPI se realizan en C. Python procesa los CSV y genera estadísticas y gráficos.

El informe distingue mediciones ejecutadas, conteos analíticos y extensiones pendientes. Incluye las campañas locales y del clúster; la ejecución multinodo y las variantes por bloques o híbridas quedan como trabajo futuro.

[1] Ballard et al. (2009), Communication-optimal Parallel and Sequential Cholesky Decomposition.

Left-looking y right-looking: ¿dónde está el paralelismo?

Left-looking: actualizar la columna que necesitamos

Al llegar a la columna j, recuperamos los aportes de las columnas ya calculadas. Primero obtenemos el pivote y luego los elementos debajo de él:

$$l_{jj} = \sqrt{a_{jj} - \sum_{k=0}^{j-1} l_{jk}^2}$$

$$l_{ij} = \frac{a_{ij} - \sum_{k=0}^{j-1} l_{ik} l_{jk}}{l_{jj}}, \qquad i > j$$

También permite paralelismo: conocido el pivote y disponibles los datos anteriores, distintos elementos l_{ij} pueden calcularse simultáneamente. Incluso una suma interna puede repartirse si luego se combinan correctamente sus resultados. En memoria distribuida debemos decidir dónde están esos datos y cuánto cuesta reunirlos.

Right-looking: actualizar la región que todavía falta

Usamos una matriz de trabajo S, inicialmente igual a A. Al llegar a j, las contribuciones anteriores ya se aplicaron:

$$l_{jj} = \sqrt{s_{jj}}, \qquad l_{ij} = s_{ij}/l_{jj} \quad (i > j)$$

Luego la columna calculada actualiza la submatriz restante:

$$s_{ik} \leftarrow s_{ik} - l_{ij} l_{kj}, \qquad i \geq k > j$$

El paralelismo aparece en esas actualizaciones: cada pareja (i, k) modifica una posición diferente y lee una columna ya terminada. Las actualizaciones de la siguiente columna deben completarse antes de factorizarla.

Ambas variantes tienen dependencias entre columnas y admiten versiones por bloques. En right-looking nos interesa repartir las actualizaciones; no ejecutar arbitrariamente todas las columnas a la vez. [1]

Experimento 1: comparar las dos variantes en C

La pregunta y el experimento

¿Right-looking es más rápido que left-looking en nuestra implementación secuencial? Implementamos ambas variantes escalares en el mismo archivo C, con double, almacenamiento por filas y el mismo compilador. No usamos BLAS ni MPI en esta comparación.

Matrices iguales para las dos variantes

El generador C, con semilla 2026, crea valores simétricos fuera de la diagonal en [−1, 1). La diagonal se define como:

$$a_{ii} = 1 + \sum_{j \neq i} |a_{ij}|$$

Esto produce una matriz simétrica con diagonal positiva y estrictamente dominante; por tanto es definida positiva. Cada llamada recibe una copia nueva de la misma matriz. Se exige un residuo relativo de reconstrucción no mayor que 10^{-12}.

Condiciones de medición

WSL / Ubuntu 26.04, AMD Ryzen 5 2600, GCC 15.2.0 y opciones -O3 -std=c11 -Wall -Wextra -Wpedantic. Ambas variantes se fijan a la misma CPU lógica. Para cada tamaño hacemos un calentamiento por variante y siete repeticiones, alternando cuál se ejecuta primero.

El tiempo incluye solo la factorización. Generación, copia y residuo quedan fuera. Conservamos las observaciones individuales y calculamos medianas y cuartiles inclusivos. La copia y validación siguen el mismo protocolo para ambas variantes, pero no garantizan un estado idéntico de caché.

n	Left (ms)	Right (ms)	Right / Left
64	0.0238	0.0226	0.95
128	0.1999	0.2680	1.34
256	1.8608	5.9010	3.17
512	16.2481	130.1355	8.01
1024	137.6874	1109.6237	8.06

Medianas de siete repeticiones por variante y tamaño. Un cociente Right / Left mayor que 1 indica que right-looking tarda más.

No es un speedup MPI.

Qué dicen los datos y qué no demuestran

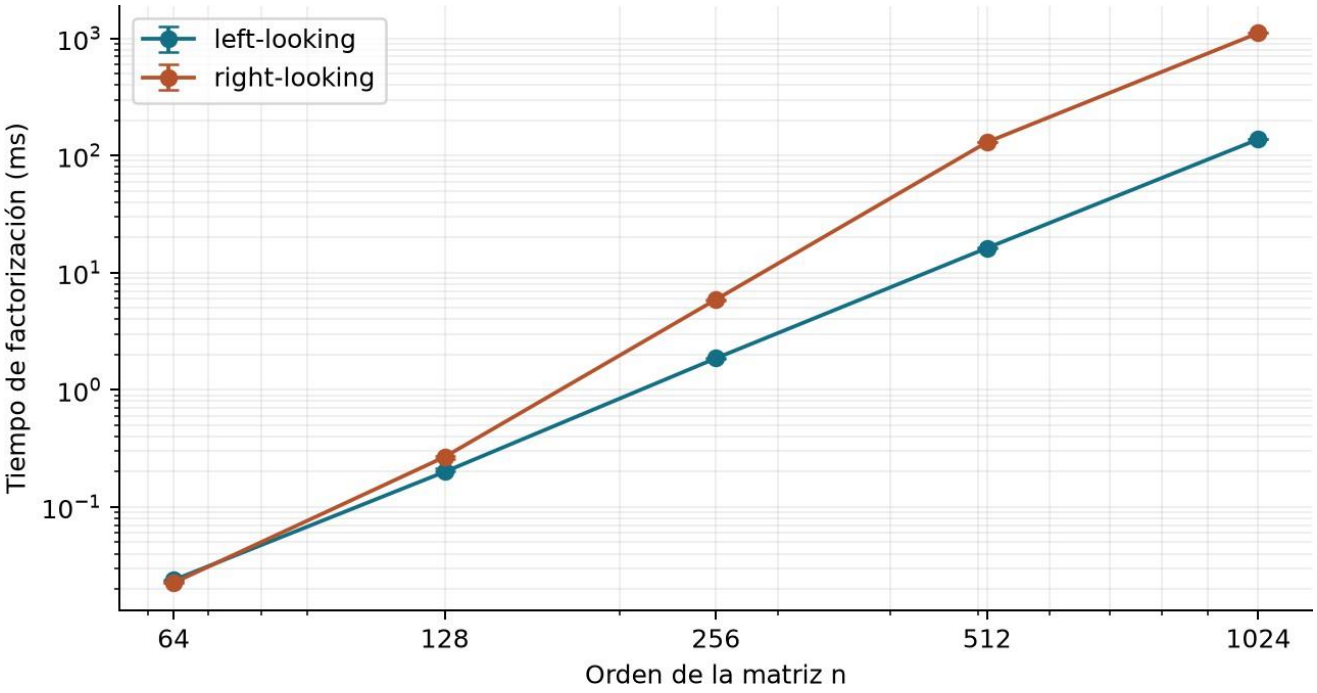


Figura 1. Comparación secuencial. Ambos ejes son logarítmicos. Los puntos indican la mediana y las barras abarcan Q1 a Q3; algunas barras son pequeñas a esta escala.

Resultado observado

La primera comparación favorece a left-looking en cuatro de los cinco tamaños ensayados. La pequeña ventaja de right-looking en n=64 cambia al aumentar la matriz: para n=1024, sus medianas son 1109,62 ms y 137,69 ms, respectivamente, de modo que right-looking tarda aproximadamente 8,06 veces más. Estos resultados descartan elegir la versión original de right-looking por su velocidad secuencial en este equipo.

La diferencia también orienta la investigación. La inspección del código muestra que left-looking utiliza recorridos contiguos por filas, mientras right-looking reutiliza datos de una columna mediante accesos separados por el ancho de la matriz. Esa diferencia es una explicación candidata, cuyo efecto todavía no está aislado. Antes de introducir MPI, corresponde comprobar si mejorar el acceso a la columna reduce el costo del kernel right-looking. La conclusión se refiere a estas implementaciones escalares y a esta campaña; la elección entre familias algorítmicas sigue abierta.

La pregunta siguiente

¿Qué pasaría si copiamos la columna activa a un vector contiguo antes de reutilizarla? Esa pregunta abre el experimento 2. Antes de agregar comunicación con MPI, comprobamos si un cambio pequeño en los accesos a datos mejora el cálculo local.

Experimento 2: cambiar el acceso, conservar el algoritmo

Hipótesis

El kernel right-looking original recorre la matriz por filas, pero lee la columna activa con saltos. Proponemos copiar esa columna a un vector contiguo y reutilizarlo durante la actualización. La hipótesis es que el ahorro en esas lecturas repetidas compense el costo de copiar.

Versión	Lectura dentro del bucle sobre k
Right original	a[k*n+j]: al aumentar k, la dirección avanza n valores.
Right + vector	column[k]: al aumentar k, la dirección avanza un valor.

$$c_k \leftarrow l_{kj}, \qquad s_{ik} \leftarrow s_{ik} - l_{ij} c_k$$

Mantenemos los bucles en el orden j, i, k, los mismos destinos y las mismas multiplicaciones y restas. El pivote y el cálculo de la columna no cambian. Esto sigue siendo Cholesky right-looking; no es aún una versión por bloques ni una implementación MPI.

Costo de la modificación

$$\text{memoria extra} = 8n \text{ bytes}, \qquad \text{copias} = \frac{n(n-1)}{2}$$

El vector reserva n valores double y se reutiliza. Para n = 1024 ocupa 8 KiB. El total de valores copiados crece como n^2, mientras que las actualizaciones mantienen su trabajo cúbico. Reservamos el vector antes del cronómetro, pero **todas las copias de columnas se incluyen en el tiempo medido**.

Comparación controlada

Volvemos a medir left, right original y right con vector en la misma campaña, sobre las mismas matrices y la misma CPU lógica. Hacemos un calentamiento por variante y doce repeticiones: las seis permutaciones del orden de ejecución se usan dos veces. Así cada variante aparece igual cantidad de veces en cada posición.

Resultados: ¿compensa copiar la columna?

n	Left (ms)	Right (ms)	Vector (ms)	Right / vector	Left / vector
64	0.0239	0.0230	0.0202	1.14	1.19
128	0.2004	0.2570	0.1725	1.49	1.16
256	1.8380	5.8159	1.3914	4.18	1.32
512	18.0215	185.6965	12.3247	15.07	1.46
1000	131.6738	95.7353	59.8545	1.60	2.20
1024	144.7909	1159.0888	76.9183	15.07	1.88

Medianas de doce repeticiones por variante y tamaño. Un cociente mayor que 1 favorece a right con vector. Cada comparación usa exclusivamente observaciones de esta segunda campaña; no mezclamos tiempos nuevos con los del experimento 1.

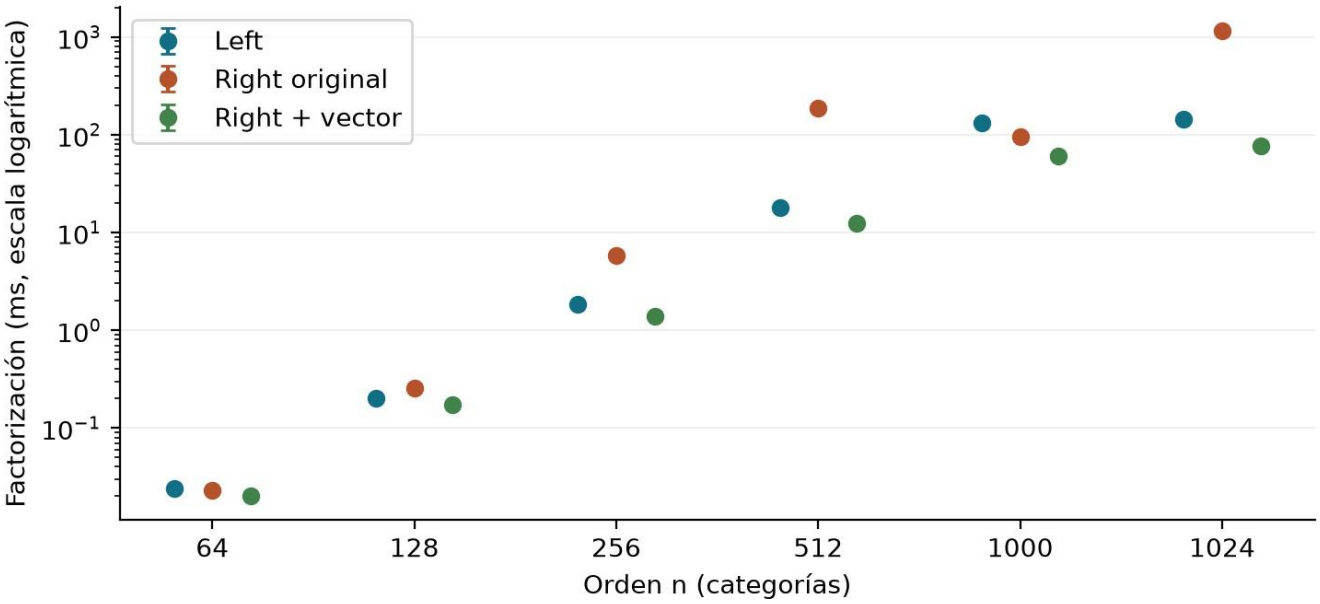


Figura 2. Medianas y barras Q1–Q3. El eje vertical es logarítmico; el horizontal muestra tamaños como categorías. Las barras representan dispersión, no intervalos de confianza.

El resultado que cambia nuestra decisión

La variante con vector obtiene la menor mediana en los seis tamaños probados, incluyendo el costo de copiar las columnas. Para n = 1024 tarda 76,92 ms, frente a 1159,09 ms de right-looking original y 144,79 ms de left-looking.

La mejora medida es de 15,07 respecto de right-looking original y de 1,88 respecto de left-looking. La interpretación y la decisión experimental se desarrollan en la página siguiente.

Interpretación y decisión después del experimento

Copiar la columna activa a un vector contiguo mejora el rendimiento de right-looking en los seis tamaños ensayados, aun contabilizando las copias dentro del tiempo de factorización. Para n=1024, la mediana disminuye de 1159,09 ms a 76,92 ms: una aceleración de 15,07 respecto de right-looking original. En la misma campaña, la variante con vector también supera a left-looking por un factor de 1,88.

La mejora conserva la corrección en los casos comprobados. Las 216 observaciones de rendimiento cumplen el límite de residuo relativo de 10^{-12}, con un máximo registrado de aproximadamente $1{,}21 \times 10^{-15}$, y las pruebas adicionales verifican casos conocidos, distintas matrices y condiciones inválidas. Esta evidencia respalda usar right-looking con vector como referencia local para continuar el proyecto.

El cambio demuestra que la organización de los accesos era una limitación importante de esta implementación. La menor sensibilidad entre n=1000 y n=1024 refuerza esa lectura, aunque faltan mediciones para separar caché, vectorización y otros efectos del compilador. El siguiente desafío es determinar si distribuir este kernel ya mejorado permite ahorrar más cálculo del que cuesta comunicar las columnas.

La pregunta siguiente

¿La ganancia de repartir estas actualizaciones alcanza para pagar el envío de columnas? El siguiente experimento MPI comparará contra la mejor referencia secuencial contemporánea. Mejorar el kernel local reduce su tiempo base y también puede exigir más trabajo para amortizar la coordinación.

II

PARTE II

Diseño MPI y primera implementación distribuida

¿Cómo repartir columnas y actualizar datos locales sin perder corrección ni introducir desequilibrio innecesario?

La razón para estudiar right-looking con MPI

Partimos de una propiedad fija de las columnas

Supongamos que cada columna tiene un proceso propietario. Queremos que los datos permanezcan en ese proceso y que el trabajo importante pueda repartirse sin migrar continuamente la matriz.

Con ese reparto	Left-looking	Right-looking
Columna j	Su dueño necesita aportes de columnas anteriores que pueden estar en otros procesos.	Su dueño encuentra la columna ya actualizada, calcula el pivote y los valores inferiores.
Trabajo repartido	Para repartir el cálculo de esa columna habría que distribuir sumas o filas y reunir resultados, o usar otra planificación.	La columna calculada se comunica; cada proceso actualiza las columnas futuras de las que ya es dueño.
Dependencias	Los aportes y el pivote deben estar disponibles.	La columna debe estar lista antes de actualizar; las actualizaciones necesarias preceden al siguiente pivote.

La tabla describe una primera organización sencilla con columnas de dueño fijo. No afirma que todas las implementaciones left-looking necesiten esa misma comunicación ni que no puedan ser competitivas.

Dónde encontramos trabajo independiente

Con right-looking, terminada la columna j, dos procesos pueden actualizar destinos diferentes usando los mismos valores recibidos. La región activa contiene muchas actualizaciones, mientras que el pivote sigue siendo parte de la cadena de dependencias.

Esta secuencia tiene una traducción directa a MPI: propietario → difusión de columna → actualizaciones locales.

[2] Netlib, Cholesky Factorization: panel y actualización distribuida right-looking.

Repartos estáticos: contiguo, cíclico y por bloques

En un reparto estático, el propietario de cada columna se fija antes de factorizar. **Round-robin por columnas es exactamente el reparto cíclico**: se asignan por turnos a los procesos 0, 1, …, P−1 y se repite.

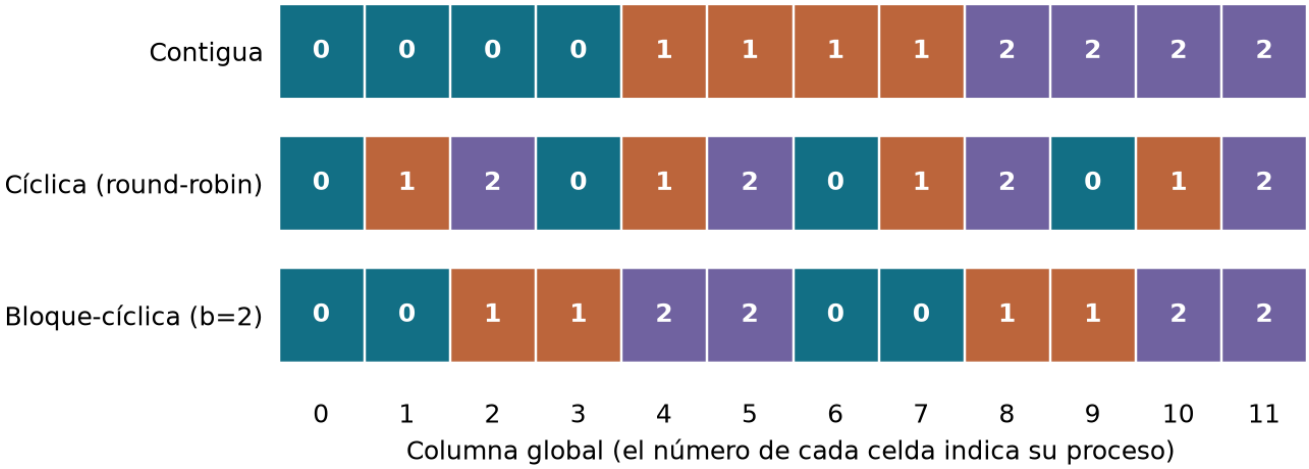


Figura 3. Ejemplo de asignación con n = 12 y P = 3. Los colores y números indican propietarios.

Contiguo: un intervalo de columnas por proceso

P0 recibe 0–3, P1 recibe 4–7 y P2 recibe 8–11. La asignación es sencilla. El problema aparece cuando termina el primer intervalo: P0 deja de tener columnas futuras que actualizar mientras otros procesos siguen trabajando. Igual cantidad de columnas no implica igual cantidad de operaciones.

Cíclico: columnas alternadas

$$\text{owner}(j) = j \bmod P$$

P0 recibe 0, 3, 6 y 9; P1 recibe 1, 4, 7 y 10; P2 recibe 2, 5, 8 y 11. Al reducirse la región activa, sus columnas quedan repartidas entre varios procesos durante más etapas. No elimina el desequilibrio ni la escasez de trabajo al final. [3]

Bloque-cíclico: repartir grupos de columnas

$$\text{owner}(j) = \lfloor j/b \rfloor \bmod P$$

Agrupa b columnas consecutivas antes de pasar al siguiente proceso. Con b = 1 recuperamos el cíclico. Con bloques mayores se busca combinar reparto y operaciones locales más grandes; el tamaño b se elegirá midiendo.

[3] ScaLAPACK Users' Guide, Two-dimensional Block-Cyclic Distribution.

Por qué el cíclico es nuestra primera hipótesis

Contemos el trabajo, sin inventar tiempos

En el kernel escalar right-looking, una columna global k recibe una actualización por cada columna anterior j. Cada una modifica sus n−k elementos inferiores. El número de actualizaciones de esa columna durante toda la factorización es:

$$U(k) = k(n-k), \qquad k = 0, \ldots, n-1$$

Sumando U(k) según los propietarios de la figura anterior obtenemos este conteo exacto para n = 12 y P = 3:

Reparto	P0	P1	P2	Máx. / media
Contigua	58	138	90	1.45
Cíclica (round-robin)	90	98	98	1.03
Bloque-cíclica (b=2)	82	106	98	1.11

Se cuentan restas de productos sobre elementos, no pivotes, divisiones, bytes ni segundos. Un valor Máx./media cercano a 1 indica mejor equilibrio de este trabajo acumulado. No representa una medición de rendimiento ni captura todas las esperas entre etapas.

Qué respalda este ejemplo

El cíclico distribuye mejor las actualizaciones que el contiguo en este caso. Para matrices densas y procesos similares, eso motiva comenzar con **propiedad estática cíclica**. Después compararemos bloque-cíclico cuando agrupemos operaciones. La decisión final dependerá del tiempo y de la comunicación medidos, no solo del conteo.

Distribución contigua no significa memoria contigua

El reparto indica qué índices globales pertenecen a un proceso. El almacenamiento local indica cómo los guarda en su RAM. Un proceso con columnas cíclicas puede empaquetarlas en un arreglo local contiguo; no necesita reservar los huecos de las columnas ajenas. [5]

Cuándo asignamos las columnas

En el esquema estático las asignamos al inicio. Durante cada etapa se comunica la columna lista y se trabaja sobre datos locales. No esperamos a un número de columna mágico para empezar a delegar. Cerca del final hay menos destinos independientes; reducir procesos o migrar datos solo convendría si su ahorro supera ese costo adicional.

[5] ScaLAPACK Users' Guide, Local Storage Scheme and Block-Cyclic Mapping.

Reparto dinámico y criterio para continuar

Qué decide una distribución dinámica

Un planificador asigna tareas durante la ejecución según qué procesos están libres. En Cholesky no puede entregar cualquier columna sin más: solo puede despachar tareas cuyas dependencias estén resueltas. Una tarea podría actualizar un bloque usando un panel ya calculado. [4]

Si el proceso elegido no tiene los datos, debe recibirlos o acceder a ellos. Además, dos tareas que modifican el mismo bloque deben respetar su dependencia o combinar sus resultados de forma segura. Por eso distinguimos la propiedad de datos de la planificación de tareas: es posible fijar una y hacer dinámica la otra.

Cuándo podría ayudar y qué cuesta

Aspecto	Estático cíclico	Dinámico
Asignación	Se conoce de antemano; no consulta una cola por tarea.	Se decide al quedar libre un trabajador y estar lista una tarea.
Desigualdad entre procesos	Un proceso lento puede retrasar etapas.	Puede asignar más tareas a trabajadores rápidos.
Datos y coordinación	Propiedad fija; comunicación del panel predecible.	Colas, mensajes, dependencias y posibles transferencias de datos.
Granularidad	Columnas o grupos con tamaño fijado.	Tareas pequeñas equilibran mejor, pero aumentan coordinación; tareas grandes reducen ese costo.

Para iniciar la implementación con matrices densas y recursos similares, una distribución estática permite definir propietarios y comunicaciones de forma sencilla y predecible. Esa simplicidad facilita comprobar la corrección y estudiar los costos antes de introducir un planificador de tareas.

La planificación dinámica sigue siendo una alternativa si aparecen esperas por heterogeneidad o por disponibilidad desigual de trabajo. Su evaluación tendría que considerar dependencias, granularidad, coordinación y movimiento de datos, además del equilibrio logrado. En esta etapa no existe una comparación temporal que permita declarar ganador a uno de los enfoques. La decisión es comenzar con el esquema estático y reconsiderar tareas dinámicas si el perfil muestra una limitación que justifique su costo y complejidad.

[4] Dorris et al., Task-Based Cholesky Decomposition: tareas listas y planificación dinámica.

¿Cuándo conviene repartir el trabajo?

En la etapa anterior mejoramos el acceso a memoria de right-looking. Ahora damos el primer paso hacia memoria distribuida: implementamos MPI en C y comparamos propiedad estática contigua y cíclica. Python analiza los tiempos y prepara gráficos; no realiza la factorización.

Hipótesis inicial

Con matrices pequeñas, comunicar cada columna puede costar más que el cálculo ahorrado. Al crecer n, esperamos que varios procesos resulten útiles. El reparto cíclico debería equilibrar mejor las actualizaciones, pero eso **no garantiza** que sea el más rápido en esta implementación.

Qué hace cada proceso

1. Asignar: repartimos las columnas antes de factorizar. Cada proceso guarda solo las suyas, empaquetadas por filas.
2. Calcular: el dueño de la columna j calcula el pivote y los elementos inferiores de esa columna.
3. Difundir: todos llaman a MPI_Bcast; el dueño envía el tramo de la columna desde j hasta n−1.
4. Actualizar: cada proceso modifica las posiciones de sus columnas k > j, usando el vector recibido. Luego avanzamos a j+1.

$$a_{ik} \leftarrow a_{ik} - l_{ij} l_{kj}, \qquad i \geq k > j$$

Los destinos pertenecen a procesos distintos: las actualizaciones de una etapa son independientes. Las columnas sucesivas siguen teniendo dependencias. Usamos comunicaciones bloqueantes y no incorporamos solapamiento ni tareas dinámicas.

La misma matriz, dos propietarios posibles

n=8, P=3	Columnas de P0	Columnas de P1	Columnas de P2
Contiguo	0, 1, 2	3, 4, 5	6, 7
Cíclico	0, 3, 6	1, 4, 7	2, 5

En contiguo, las primeras n mod P tareas reciben una columna adicional. En cíclico, owner(k)=k mod P. Incluso si P > n, los procesos sin columnas participan en las comunicaciones. El reparto no cambia durante la factorización.

MPI_Bcast distribuye un mensaje desde el dueño a todos los procesos del comunicador. Fuente: documentación oficial de Open MPI.

Cómo medimos y qué comparamos

Decisión	Protocolo ejecutado
Equipo	Un equipo, WSL2 / Ubuntu 26.04, AMD Ryzen 5 2600; Open MPI 5.0.10 y GCC 15.2.0.
Compilación	-O3 -std=c11 -Wall -Wextra -Wpedantic; double, sin BLAS.
Tamaños y procesos	n = 64, 256, 512, 1024; P = 1, 2, 3, 4. Un proceso por núcleo distinto, según la topología expuesta por WSL.
Repeticiones MPI	Un calentamiento por reparto y 8 repeticiones medidas. Contiguo y cíclico alternan el orden. El orden de P rota entre tamaños.
Referencias secuenciales	Left, right original y right con vector. Seis repeticiones antes y seis después de MPI para cada tamaño; misma semilla 2026.

El tiempo principal incluye cálculo y comunicación

$$T_P = \max_{r=0,\ldots,P-1} (t_{r,\text{fin}} - t_{r,\text{inicio}})$$

Una barrera coordina el inicio. Cada proceso mide su duración con MPI_Wtime; al finalizar tomamos el máximo. No restamos marcas de reloj de procesos diferentes. El cronómetro incluye el armado del vector, las difusiones y las actualizaciones; excluye generación, reserva, distribución inicial, reunión del factor y comprobaciones. Tampoco incluye iniciar mpirun.

Por tanto, medimos el kernel de factorización, no el tiempo de la aplicación completa. El proceso 0 genera y reúne matrices completas para este banco de pruebas; esa preparación también limita la escalabilidad futura.

Dos aceleraciones responden preguntas distintas

$$S_{\text{MPI}} = \frac{\widetilde{T}_{\text{MPI},1}}{\widetilde{T}_{\text{MPI},P}}, \qquad S_{\text{util}} = \frac{\min_v \widetilde{T}_{v,\text{seq}}}{\widetilde{T}_{\text{MPI},P}}$$

La primera compara un reparto consigo mismo al aumentar P. La segunda comprueba si supera al mejor secuencial de esta campaña; esa es la relevante para decidir si nos conviene usar MPI. Reportamos medianas y cuartiles inclusivos. Los cuartiles describen dispersión; no son intervalos de confianza.

MPI_Wtime mide tiempo local. Fuente: documentación oficial de Open MPI.

Resultado: más procesos no siempre ayudan

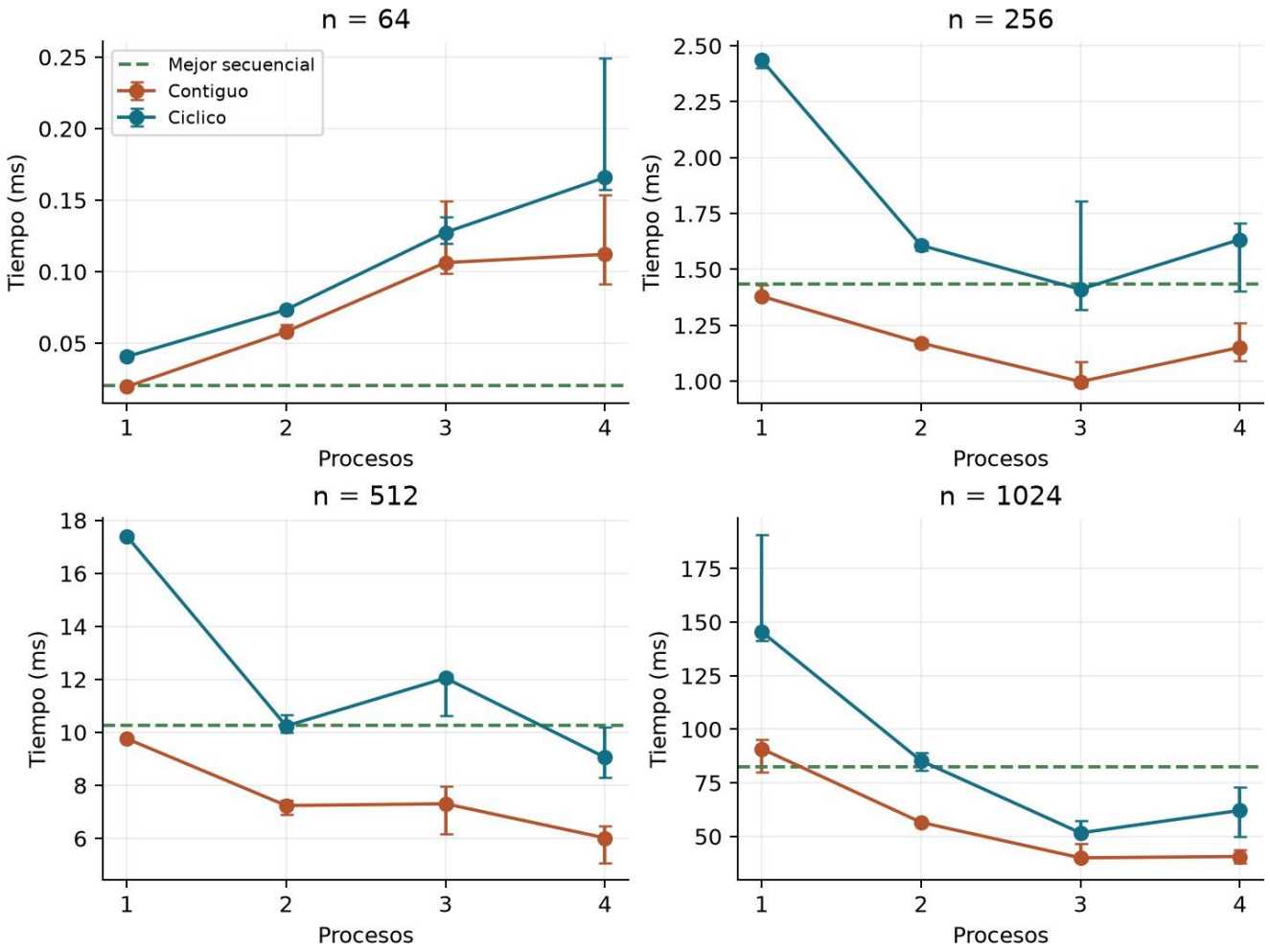


Figura 1. Medianas de 8 repeticiones por configuración; barras Q1-Q3. La línea verde muestra la mejor mediana secuencial contemporánea. Cada panel tiene su propia escala vertical.

Lo que vemos

n=64: usar más de un proceso aumenta el tiempo en ambos repartos. Hay demasiado poco trabajo para amortizar la coordinación.
n=256 y n=512: el contiguo ya obtiene mejoras útiles; el cíclico presenta resultados cercanos al secuencial o más lentos en varias configuraciones.
n=1024: ambos repartos mejoran al pasar de 1 a 3 procesos. Agregar un cuarto no reduce la mediana en esta campaña.

Contiguo y cíclico comparten el código de factorización y las operaciones matemáticas. Sin embargo, el reparto cambia los índices, los accesos al vector difundido y el código que puede optimizar el compilador. Esta prueba compara las implementaciones completas, no aísla exclusivamente el balance de carga.

¿Cuándo empieza a convenir MPI?

n	Mejor sec. (ms)	Mejor MPI P≥2	MPI (ms)	S útil
64	0,021	Contiguo, P=2	0,058	0,35
256	1,434	Contiguo, P=3	0,998	1,44
512	10,265	Contiguo, P=4	6,019	1,71
1024	82,654	Contiguo, P=3	40,028	2,06

Selección descriptiva de la menor mediana entre configuraciones probadas, no una garantía de que siempre sea la mejor. Right con vector fue la mejor referencia secuencial en los cuatro tamaños.

Respuesta provisional para este equipo

Con n=64 no conviene paralelizar este kernel. El primer tamaño probado con mejora clara del contiguo es n=256. Para n=1024 y P=3, el contiguo tarda 40,03 ms frente a 82,65 ms del secuencial: S útil = 2,06.

No encontramos un umbral exacto. Faltan tamaños intermedios y campañas independientes. Tampoco suponemos que la conveniencia sea monótona: caché, reparto, sistema operativo y red pueden cambiar el resultado. P=3 y P=4 contiguos tienen cuartiles que se superponen en n=1024; no declaramos una superioridad concluyente de tres procesos.

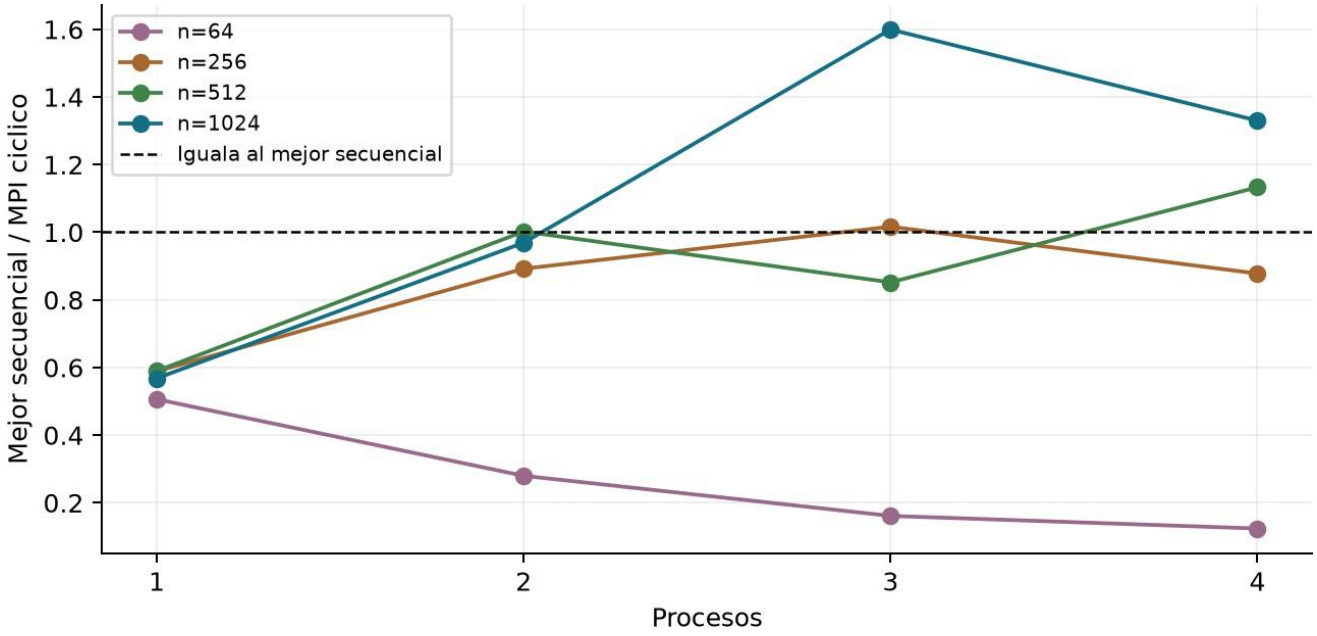


Figura 2. Aceleración útil del cíclico. Cerca de 1, una pequeña diferencia de medianas no basta para afirmar una mejora robusta. La evaluación entre nodos queda pendiente.

¿Por qué no ganó el cíclico?

La hipótesis de mejor reparto sí se cumple para el **número de actualizaciones**. Para n=1024 y P=3, los conteos exactos por proceso son:

Reparto	P0	P1	P2
Contiguo	46 434 993	86 086 814	46 434 993
Cíclico	59 652 153	59 652 153	59 652 494

Pero el tiempo depende también de cuánto cuesta cada actualización y cuándo queda disponible. En el contiguo, el vector difundido se lee en posiciones consecutivas. En el cíclico, sus índices avanzan de P en P, aunque los destinos locales estén empaquetados. El cálculo de índices también difiere.

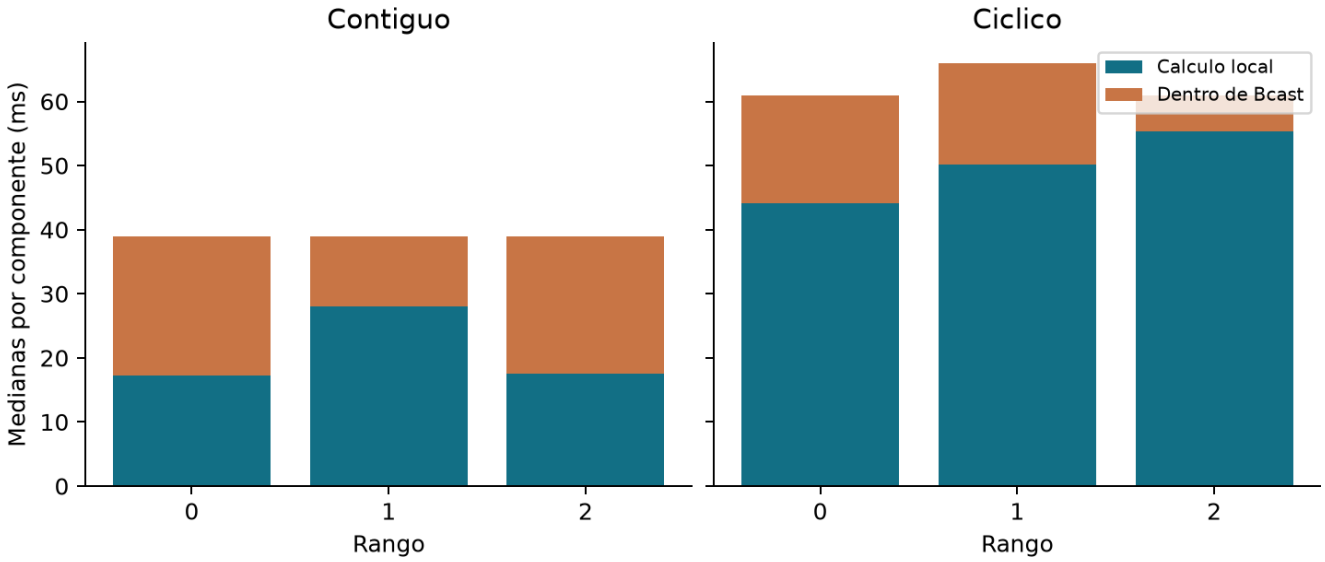


Figura 3. Perfil separado para n=1024, P=3, con 3 repeticiones e instrumentación interna. Se apilan medianas de componentes; su suma no es necesariamente la mediana del total. Bcast incluye transferencia y espera por otros procesos: no mide únicamente red.

Conclusiones

La primera implementación MPI obtiene beneficios frente al mejor secuencial contemporáneo en varios tamaños, pero favorece al reparto contiguo en todas las combinaciones de tamaño y cantidad de procesos probadas. Para n=1024, el contiguo con tres procesos alcanza 40,03 ms frente a 82,65 ms secuenciales, con aceleración útil de 2,06. En n=64, la coordinación agregada supera el ahorro de cálculo y conviene conservar la ejecución secuencial.

El resultado obliga a distinguir equilibrio de operaciones y velocidad efectiva. El cíclico reparte mejor las actualizaciones, pero incluso con P=1 tarda 145,49 ms frente a 90,90 ms del contiguo, cuando ambos poseen las mismas columnas. Esa diferencia demuestra que el desbalance entre procesos no explica por sí solo la desventaja del cíclico; también intervienen los caminos de acceso, indexación y ejecución local.

Conservamos el contiguo como referencia práctica de esta etapa y revisamos el kernel local del cíclico antes de descartarlo. La campaña demuestra una posibilidad de aceleración del kernel, con generación, distribución inicial y reunión fuera del reloj; todavía no establece el beneficio del programa completo ni un umbral exacto de conveniencia de MPI.

¿Y si ambos repartos leyeran datos contiguos?

En el experimento anterior, el cíclico equilibraba mejor el número de operaciones, pero tardaba más. Había una pista: incluso con un solo proceso, cuando ambos repartos poseen las mismas columnas, sus tiempos eran distintos. Antes de descartar el reparto cíclico, decidimos reducir el costo de acceso e indexación del cálculo local.

Hipótesis

Después de recibir la columna j, cada proceso copia los valores que necesita a un vector ordenado como sus columnas propias. Esperamos que reutilizar ese vector compense el costo de copiarlo, especialmente en el cíclico. La copia se incluye en el tiempo: no damos por hecho que sea beneficiosa.

Cuánto cuesta y qué se conserva

Cada proceso reserva tantos doubles adicionales como columnas propias: ocho bytes por columna local en este equipo. La memoria adicional máxima por proceso es:

$$M_{\text{extra}} \leq 8\left\lceil \frac{n}{p} \right\rceil \text{ bytes}$$

Durante toda la factorización se copian n(n−1)/2 valores entre todos los procesos. Es trabajo cuadrático adicional frente al cálculo cúbico de Cholesky; eso por sí solo no garantiza una mejora para tamaños pequeños.

No cambian los propietarios, las dependencias, el orden de las operaciones aritméticas ni las llamadas a MPI_Bcast. El bucle interno accede ahora a datos consecutivos en ambos repartos. La asignación de índices fuera de ese bucle todavía difiere.

Un experimento con cuatro variantes

Reparto	Lectura original	Vector local
Contiguo	Referencia anterior	Nueva variante
Cíclico	Referencia anterior	Nueva variante

Cómo evitamos mezclar campañas

Volvimos a medir las cuatro variantes en el mismo ejecutable y sobre las mismas matrices. Los cocientes de esta etapa usan únicamente tiempos nuevos; no dividimos tiempos actuales por los del PDF anterior. La generación y todas las factorizaciones están en C. Python valida los CSV y obtiene estadísticas y gráficos.

Configuración	Protocolo ejecutado
Entorno	WSL2 / Ubuntu 26.04, AMD Ryzen 5 2600, Open MPI 5.0.10, GCC 15.2.0; -O3, double, sin BLAS.
Tamaños y procesos	n=64,256,512,1024 y P=1,2,3,4. Un proceso por núcleo distinto según la topología expuesta por WSL.
Orden y repeticiones	Un calentamiento por variante; ocho repeticiones medidas. Cada variante ocupa cada posición dos veces. Se equilibran también las parejas consecutivas dentro de cada repetición.
Tiempo MPI	Máximo de las duraciones locales, con inicio coordinado por barrera. Incluye copia del vector, cálculo y difusión.
Fuera del cronómetro	Reserva de memoria, generación, distribución inicial, reunión, validación y arranque de MPI.
Referencia secuencial	Left, right original y right con vector: seis ejecuciones antes y seis después de MPI para cada n.

Qué reportamos

$$G_{\text{copia}} = \frac{\widetilde{T}_{\text{original}}}{\widetilde{T}_{\text{local}}}, \qquad S_{\text{util}} = \frac{\widetilde{T}_{\text{mejor secuencial}}}{\widetilde{T}_{\text{MPI}}}$$

Un cociente mayor que 1 indica una mejora. Usamos medianas y cuartiles inclusivos Q1-Q3. Las barras de dispersión no son intervalos de confianza; ocho repeticiones en una campaña no sustituyen repeticiones independientes en otros momentos.

Resultado: la copia ayuda sobre todo al cíclico

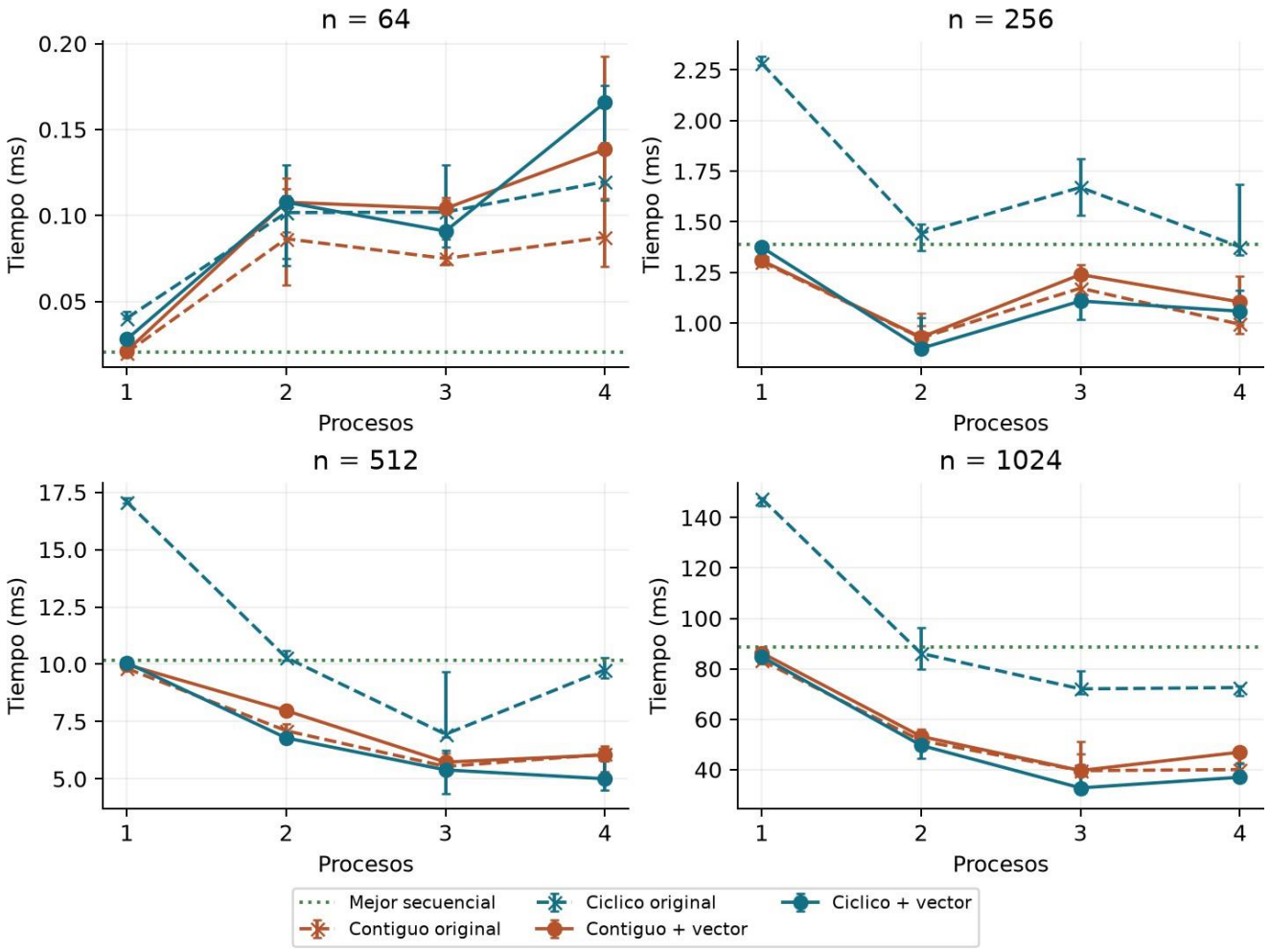


Figura 1. Medianas de ocho repeticiones; barras Q1-Q3. Líneas discontinuas: acceso original. Líneas continuas: vector local. La referencia secuencial es la mejor mediana de las tres variantes secuenciales de esta campaña. Cada panel tiene su propia escala.

La hipótesis se cumple con límites

n=64: todos los casos con P>1 son más lentos que el mejor secuencial; agregar una copia no resuelve la falta de trabajo útil.
n≥256: el cíclico con vector tiene menor mediana que el cíclico original en todos los P probados.
Contiguo: copiar no aporta una mejora sistemática; su lectura original ya era consecutiva y en varios casos la copia agrega costo.

Entre las nuevas variantes, el cíclico presenta menor mediana que el contiguo para P=2,3,4 y n≥256. Sin embargo, varias distribuciones de tiempos se superponen: esto respalda una elección provisional, no una superioridad universal ni una prueba de significación estadística.

El caso n=1024, explicado con números

P	Contiguo original	Contiguo + vector	Cíclico original	Cíclico + vector
1	83,28	86,33	147,55	84,81
2	51,49	53,17	86,11	49,64
3	39,62	39,66	72,04	32,77
4	40,03	46,93	72,57	36,99

Todos los tiempos están en milisegundos. Mediana del mejor secuencial: 88,62 ms, correspondiente a right con vector.

¿Se amortizó la copia?

Con P=3, el cíclico pasa de 72,04 ms a 32,77 ms: mejora 2,20 veces respecto de su versión original, incluyendo el tiempo de copiar. Frente al mejor secuencial, la aceleración útil es 2,70.

El control con un solo proceso

Antes de copiar, P=1 da 83,28 ms en contiguo y 147,55 ms en cíclico. Con el vector local, quedan en 86,33 y 84,81 ms, con cuartiles superpuestos. La gran diferencia previa se redujo: el reparto entre procesos no podía explicar esa diferencia inicial.

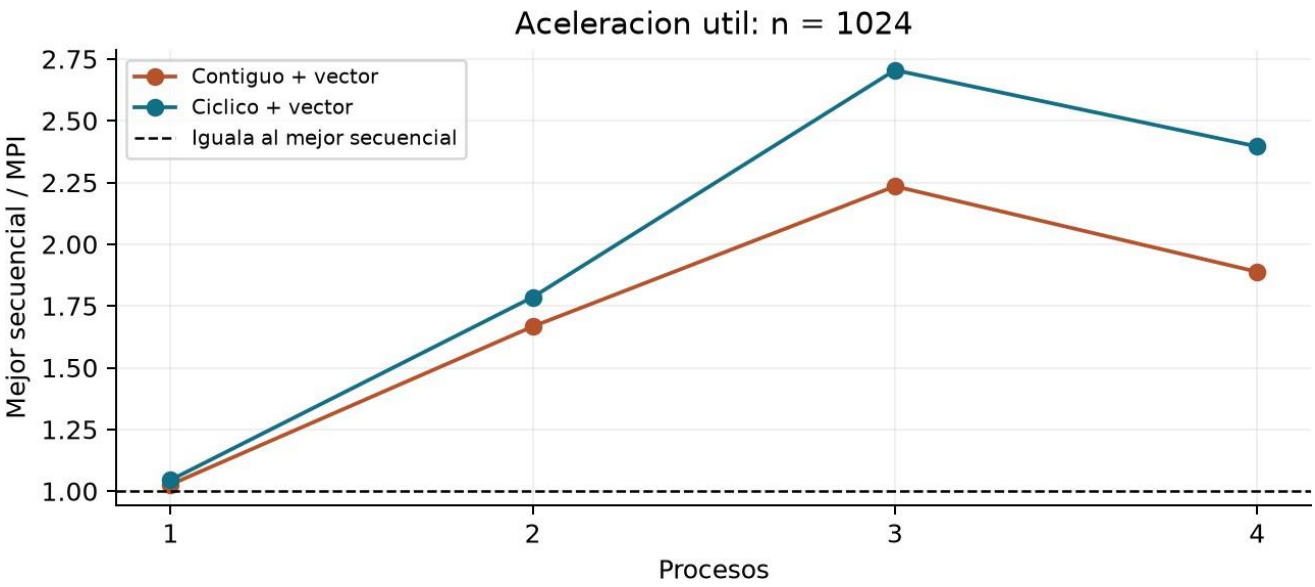


Figura 2. Aceleración útil de las nuevas variantes. Tres procesos cíclicos dieron la menor mediana para n=1024; sus cuartiles se superponen con los de cuatro procesos y con el contiguo de tres. Debemos repetir antes de afirmar cuál es la mejor configuración de forma estable.

Qué decidimos y cuál es la próxima pregunta

Qué podemos concluir

El vector local permite al reparto cíclico aprovechar mejor su distribución del trabajo. Para n≥256, reduce su mediana frente al cíclico original en todos los valores de P ensayados. En n=1024 y P=3, el tiempo pasa de 72,04 ms a 32,77 ms, una mejora de 2,20 que incluye la copia. Frente al mejor secuencial de la misma campaña, la aceleración útil alcanza 2,70.

El control con un proceso refuerza la interpretación: la diferencia previa de 83,28 ms frente a 147,55 ms entre contiguo y cíclico se reduce a 86,33 ms frente a 84,81 ms con vector. La mejora del acceso y la indexación, acompañada por la vectorización registrada por el compilador, es compatible con la reducción del costo local. La campaña no separa cuánto aporta cada mecanismo.

La copia no mejora sistemáticamente al contiguo, que ya leía consecutivamente los valores utilizados. Adoptamos el cíclico con vector como candidato principal y mantenemos el contiguo original como referencia. Las distribuciones de tiempos de algunas configuraciones se superponen, por lo que la elección requiere confirmación; para matrices pequeñas sigue siendo preferible el secuencial en las condiciones medidas.

¿En qué tamaño empieza a compensar esta versión?

Para responder mejor, el próximo paso razonable es probar tamaños intermedios entre 64 y 256, repetir campañas independientes y comparar pocos valores de P. Así evitamos elegir un umbral por una sola mediana.

III

PARTE III

Cruce de rendimiento y comparación de algoritmos MPI

¿En qué tamaños MPI empieza a ayudar y qué variante paralela conviene conservar?

¿Cuál variante nos conviene paralelizar?

Hasta ahora habíamos elegido right-looking por su organización natural de las actualizaciones, pero eso era una razón de diseño. Para responder con evidencia implementamos también left-looking con MPI y comparamos ambas variantes bajo las mismas condiciones.

La dependencia que comparten

En ambas variantes, la columna j solo puede finalizar cuando están disponibles las contribuciones que la afectan. Right-looking no elimina esa dependencia ni permite calcular arbitrariamente todas las columnas a la vez.

Dónde aparece el trabajo paralelo

Variante	Antes de calcular j	Trabajo repartido después
Left-looking	Reúne desde las columnas 0,...,j-1 las contribuciones necesarias para j.	Con el pivote listo, calcula en paralelo distintos elementos Lij de la columna actual.
Right-looking	Las columnas anteriores ya aplicaron sus aportes sobre la matriz de trabajo.	Con j lista, actualiza en paralelo muchas posiciones de las columnas futuras.

$$l_{ij} = \frac{a_{ij} - \sum_{k<j} l_{ik} l_{jk}}{l_{jj}} \quad \text{(left)}$$

$$s_{ik} \leftarrow s_{ik} - l_{ij} l_{kj}, \quad i \geq k > j \quad \text{(right)}$$

Una formulación precisa

Left-looking reúne el pasado para construir la columna actual. Right-looking propaga la columna actual hacia el futuro. Ambos admiten paralelismo; right-looking expone un conjunto amplio de actualizaciones sobre la región restante. En versiones por tareas se puede priorizar la próxima columna y continuar otras actualizaciones como look-ahead.

Nuestra versión right-looking es sincronizada por columnas: difunde j y cada proceso actualiza sus columnas propias antes de la etapa siguiente. Por eso esta comparación evalúa dos implementaciones concretas, no el máximo paralelismo teórico posible de cada familia.

Cómo hicimos comparable el experimento

La misma propiedad de datos

Ambas variantes usan distribución cíclica estática. Cada proceso almacena sus columnas en memoria local contigua y recibe exactamente la misma matriz SPD. La distribución inicial y la reunión final son idénticas y quedan fuera del cronómetro del kernel.

Left-looking MPI	Right-looking MPI
Cada proceso calcula sumas parciales con sus columnas anteriores.	El propietario calcula la columna j.
MPI_Reduce reúne el pivote y los elementos inferiores en el propietario de j.	MPI_Bcast difunde el tramo terminado de j.
El propietario guarda la nueva columna; se difunde el pivote.	Cada proceso actualiza sus columnas futuras con vector local contiguo.

Diferencia de coordinación

Por columna, left usa una reducción escalar, una reducción vectorial -salvo en la última columna- y una difusión escalar: hasta 3n-1 colectivas. Right usa una difusión vectorial por columna: n colectivas. Los volúmenes exactos dependen de la implementación de MPI, pero left introduce más puntos de coordinación.

Protocolo

Decisión	Valor
Tamaños y procesos	n=64,128,256,512,1024; P=1,2,3,4.
Repeticiones	Un calentamiento y ocho mediciones por algoritmo y configuración. Orden left/right alternado.
Entorno	WSL2, AMD Ryzen 5 2600, Open MPI 5.0.10, GCC 15.2.0, -O3, double, sin BLAS.
Tiempo	Máximo de las duraciones locales tras una barrera. Incluye cálculo y colectivas; excluye Scatterv, Gatherv y validación.
Correctitud	Residuo relativo <=10^-12 y comparación con el factor right secuencial.

Resultado: right fue más rápido en todos los casos

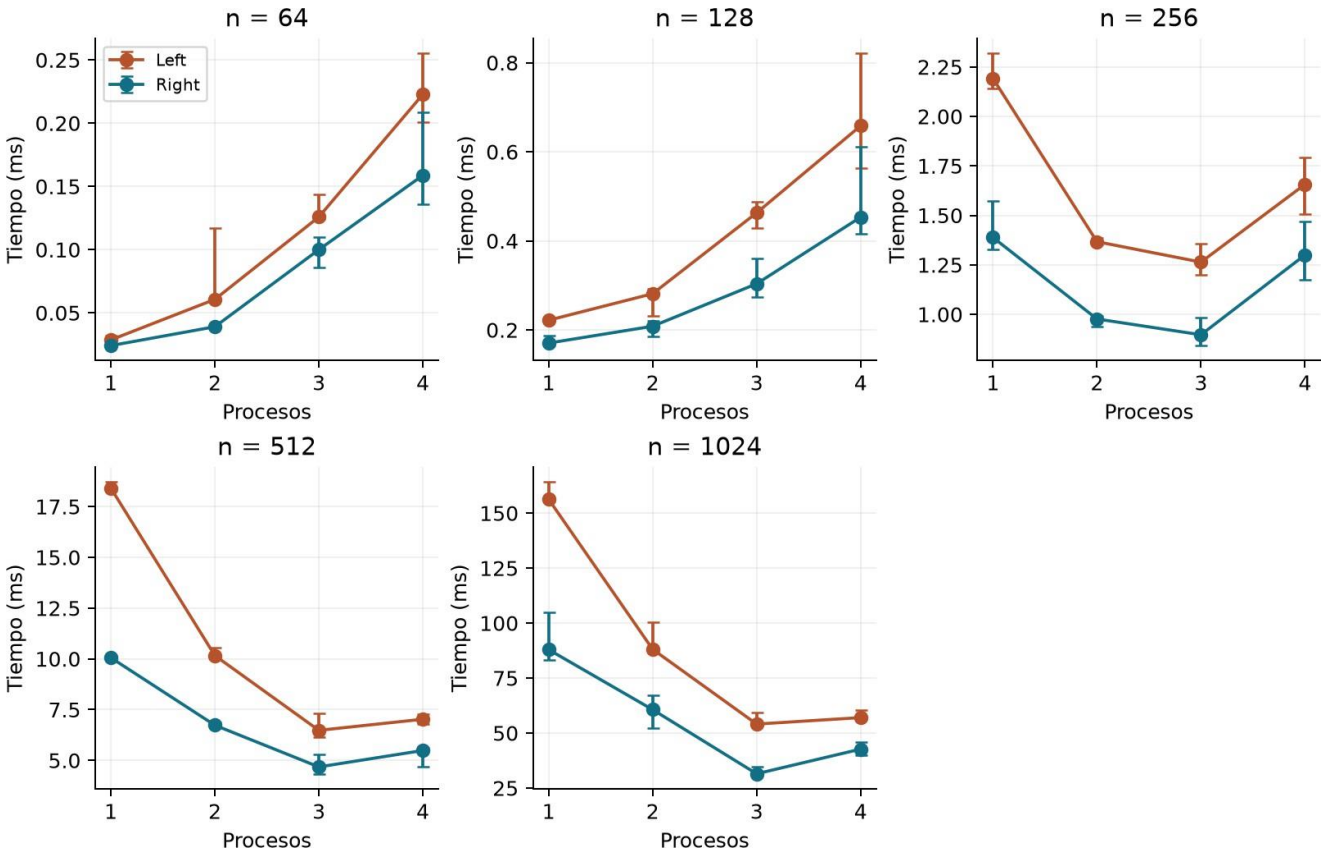


Figura 1. Medianas de ocho repeticiones; barras Q1-Q3. Se usa una escala propia por panel.

n	P	Left (ms)	Right (ms)	Left / Right
64	1	0,03	0,02	1,19
128	1	0,22	0,17	1,30
256	3	1,26	0,90	1,41
512	3	6,48	4,68	1,38
1024	3	54,08	31,48	1,72

La tabla usa, para cada n, el P que minimizó la mediana de right. No selecciona P por separado para favorecer a left. En las 20 configuraciones completas, right tuvo menor mediana.

Caso principal: n=1024

Con tres procesos, left tardó 54,08 ms y right 31,48 ms: left fue 1,72 veces más lento. Respecto de su propio P=1, left alcanzó 2,89x y right 2,79x. Ambos mejoraron al paralelizar y ambos empeoraron al pasar de tres a cuatro procesos.

Por qué right ganó en nuestra implementación

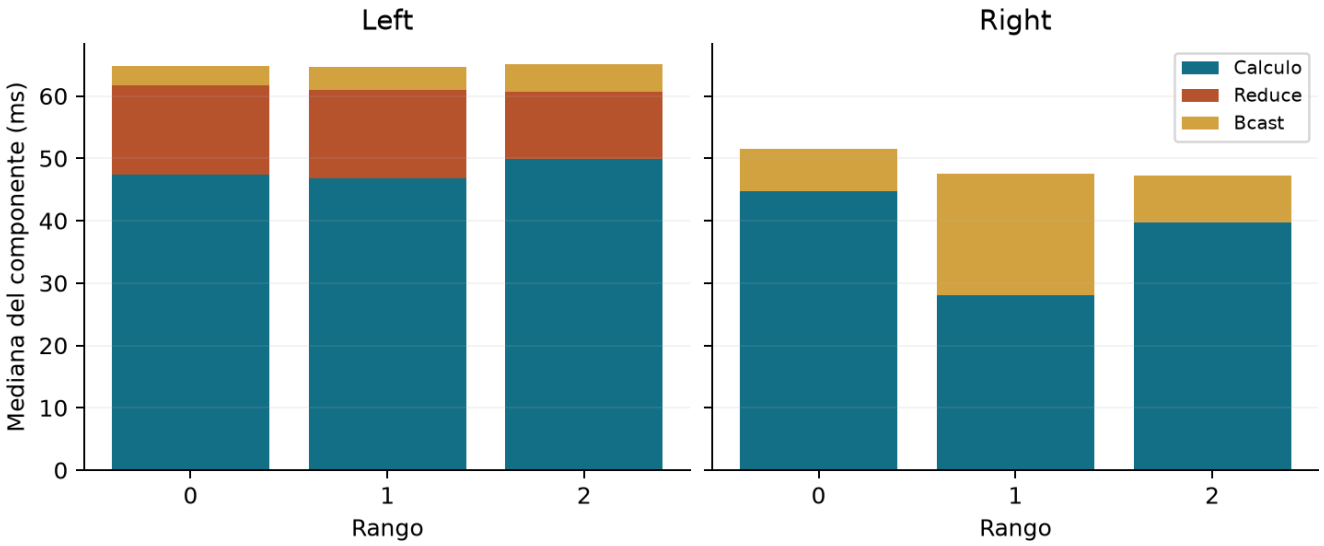


Figura 2. Perfil separado de n=1024, P=3, cuatro repeticiones por algoritmo.

La comunicación de left es más fragmentada

Left necesita reunir sumas parciales en cada columna. Aunque reparte bien los productos locales, paga reducciones vectoriales y escalares además de difundir cada pivote. Right difunde la columna terminada una vez y realiza actualizaciones locales sobre los datos que cada proceso posee.

En el perfil, los tres rangos de left acumulan aproximadamente entre 10,80 y 14,29 ms dentro de MPI_Reduce, además de sus difusiones. Right no usa reducciones en el kernel. Esta observación explica parte de la diferencia, pero el tiempo dentro de una colectiva mezcla transferencia y espera.

Conclusión

La comparación directa respalda continuar con right-looking MPI con vector local: obtiene menor mediana que left-looking MPI en las veinte configuraciones ensayadas. Para n=1024 y P=3, right-looking tarda 31,48 ms y left-looking 54,08 ms; el cociente de 1,72 expresa la ventaja temporal de right-looking bajo el mismo reparto y entorno.

Ambas variantes se benefician del paralelismo. En ese tamaño, left-looking alcanza una aceleración de 2,89 respecto de su propio P=1, ligeramente mayor que el 2,79 de right-looking, aunque conserva un tiempo absoluto superior. El resultado muestra por qué la escalabilidad respecto de una base propia debe acompañarse del tiempo final: una mayor aceleración relativa puede coexistir con una ejecución más lenta.

La implementación left-looking introduce hasta 3n−1 colectivas frente a las n difusiones de right-looking. El perfil registra un costo apreciable dentro de las reducciones y respalda la coordinación como parte de la diferencia, sin separar transferencia y espera. La decisión se limita a estas implementaciones escalares, distribución cíclica y condiciones locales. Right-looking queda seleccionado por rendimiento observado, además de su conveniencia de diseño.

¿Cuándo conviene usar más procesos?

MPI resulta conveniente cuando el ahorro conseguido al repartir el cálculo supera los costos adicionales del camino paralelo dentro de la tarea que interesa medir. El modelo de trabajo dependiente, actualizaciones repartibles y comunicación orienta esa evaluación, pero no determina por sí solo un tamaño de cruce.

La decisión práctica debe comparar MPI con la mejor referencia secuencial medida en condiciones equivalentes. La comparación con MPI P=1 responde otra pregunta: cuánto escala el propio programa paralelo. Si el objetivo es reducir la espera hasta que termine el comando, también deben medirse preparación, validación, arranque y cierre según los límites de ese reloj. Esta distinción fija el criterio de los experimentos siguientes y evita obtener aceleraciones favorables a partir de una referencia secuencial que ya fue superada.

Repartir trabajo ahorra cálculo por proceso, pero agrega comunicación, coordinación y posibles esperas. Un modelo orientativo, sin solapamiento, es:

$$T_P \approx T_{\text{dep}} + \frac{T_{\text{upd}}}{P} + T_{\text{comm}} + T_{\text{espera}}$$

Tdep representa trabajo dependiente no repartido;

Tupd, las actualizaciones repartibles.

En este modelo, usar P procesos compensa frente al mismo algoritmo secuencial cuando:

$$T_{\text{upd}}\left(1 - \frac{1}{P}\right) > T_{\text{comm}} + T_{\text{espera}}$$

Para un mensaje de m bytes, el modelo α + βm distingue el costo de iniciar el envío y el costo por volumen. Un broadcast depende además de P y de la implementación MPI. [1]

La comparación que realmente decide

$$T_{\text{base}}(n) = \min(T_{L,1}(n), T_{R,1}(n), T_{R_v,1}(n))$$

$$S_{\text{util}}(n, P) = \frac{T_{\text{base}}(n)}{T_{R_v,P}(n)}$$

Rv indica right con vector. Conviene MPI si reduce de forma repetible el tiempo frente a esa base. También compararemos el mismo kernel MPI con P = 1 y P mayor para aislar su escalabilidad. Si importa el tiempo completo de una tarea, debemos sumar arranque y distribución inicial.

Para n = 1024, el experimento 2 deja una referencia local de 76,92 ms. Una versión MPI tendría que mejorar ese tiempo total de factorización en condiciones comparables. No usaremos los 1159,09 ms de right original para presentar un speedup artificialmente favorable.

Cómo obtendremos el umbral

Para cada n probaremos P = 1, 2, 3, 4 y otros valores permitidos, con varias ejecuciones. Elegiremos el menor tiempo estable; si la diferencia es pequeña frente a la dispersión, preferiremos menos recursos. El primer n favorable será un cruce observado para esa configuración, no una constante universal.

¿Cuándo empieza a compensar MPI?

Después de mejorar el cíclico con un vector local, observamos que MPI podía ganar con matrices grandes y perder con pequeñas. Faltaba precisar la transición. Este experimento conserva el código C del anterior y cambia únicamente el plan de medición.

La pregunta concreta

Entre n=64 y n=256, ¿qué tamaños permiten al cíclico con vector superar al mejor secuencial de manera consistente con dos o tres procesos?

Protocolo fijado antes de medir

Decisión	Ejecución
Tamaños	64, 96, 128, 160, 192, 224 y 256.
Rondas	Tres rondas con lanzamientos separados en la misma sesión. Orden de tamaños ascendente, descendente y rotado; orden de P también rotado.
Repeticiones	Ocho tiempos por variante MPI y configuración, tras calentamiento. Doce tiempos por variante secuencial: seis antes y seis después.
Entradas	Mismo generador SPD en C, misma semilla 2026 y mismo n para todas las variantes.
Entorno	WSL2, AMD Ryzen 5 2600, GCC 15.2.0 y Open MPI 5.0.10. -O3; un proceso por núcleo distinto según WSL.

Criterio conservador de mejora

$$S_r = \frac{\min_v \widetilde{T}_{v,\mathrm{seq},r}}{\widetilde{T}_{\mathrm{MPI},r}}$$

$$S_r > 1.10 \quad \text{y} \quad Q_{3,\mathrm{MPI},r} < Q_{1,\mathrm{seq},r}, \qquad r = 1,2,3$$

Exigimos aceleración superior a 1,10 y separación entre los cuartiles centrales en las tres rondas. Los cuartiles secuenciales corresponden a la variante con menor mediana en esa ronda. El margen del 10 % es una decisión práctica explícita, no una constante de Cholesky.

Esta regla es descriptiva: Q1-Q3 no es un intervalo de confianza y su separación no demuestra significación estadística. Las tres rondas comparten equipo y sesión; no representan días, máquinas o condiciones independientes.

La transición observada entre 64 y 256

n	S con P=2	Criterio	S con P=3	Criterio
64	0,27 - 0,51	0/3	0,15 - 0,22	0/3
96	0,53 - 0,69	0/3	0,38 - 0,39	0/3
128	0,72 - 0,88	0/3	0,66 - 0,77	0/3
160	0,72 - 0,96	0/3	0,57 - 0,87	0/3
192	0,87 - 0,91	0/3	0,70 - 0,93	0/3
224	0,73 - 0,99	0/3	0,83 - 1,06	0/3
256	1,13 - 1,43	2/3	1,12 - 1,49	3/3

Cada intervalo muestra el mínimo y máximo de las tres aceleraciones calculadas por ronda. No es un intervalo de confianza. “Criterio” cuenta las rondas que cumplen simultáneamente ambas condiciones.

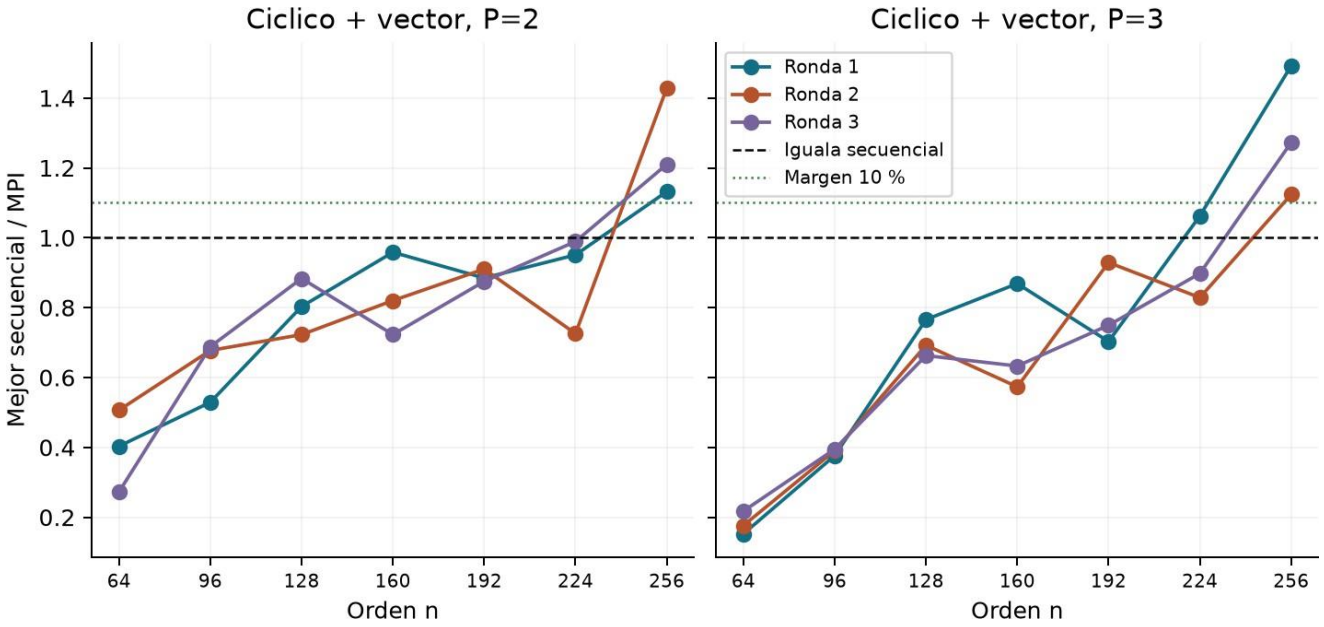


Figura 1. Aceleración del cíclico con vector frente al mejor secuencial contemporáneo. Por encima de 1 gana MPI; por encima de 1,10 supera el margen elegido, aunque todavía debe cumplir la condición de cuartiles.

Qué muestran las repeticiones

n≤192: MPI con dos y tres procesos tuvo mayor mediana que el secuencial en las tres rondas.
n=224: tres procesos ganaron apenas en una ronda y perdieron en las otras dos.
n=256: ambos valores de P ganaron por mediana en las tres rondas; solo P=3 cumplió también el criterio completo en las tres.

Por tanto, n=256 con P=3 es el **primer punto de la grilla probada** que satisface nuestra regla. No medimos un umbral exacto y no demostramos que todo n mayor que 256 tenga el mismo comportamiento.

Por qué dos y tres procesos no reciben la misma conclusión

Ronda	Secuencial (ms)	MPI P=2 (ms)	MPI P=3 (ms)
1	1,431	1,264	0,959
2	1,388	0,972	1,235
3	1,398	1,156	1,099

Medianas para n=256. Right con vector fue la referencia secuencial elegida en estas tres comparaciones. No se mezclan rondas para calcular una aceleración única.

Una mediana favorable puede convivir con dispersión

En la primera ronda, dos procesos tienen aceleración 1,13, superior a 1,10. Sin embargo, su Q3 es 1,452 ms y el Q1 secuencial es 1,380 ms. Los intervalos centrales se superponen y la configuración no pasa nuestra segunda condición.

Con tres procesos, las tres rondas cumplen ambas condiciones. Sus aceleraciones son 1,49, 1,12, 1,27. Eso respalda una mejora consistente dentro de esta sesión, pero no demuestra que tres procesos siempre sean mejores que dos: en la segunda ronda, P=2 tuvo menor mediana que P=3.

Conclusión

En la grilla local ensayada, n=256 con tres procesos es el primer punto que cumple en las tres rondas el criterio prefijado de mejora: aceleración mayor que 1,10 y separación de los cuartiles centrales respecto del mejor secuencial contemporáneo. Las aceleraciones de 1,49, 1,12 y 1,27 respaldan una ventaja consistente dentro de esta sesión.

Con dos procesos también hay una mediana favorable en las tres rondas para n=256, pero el criterio completo se cumple en dos. En la ronda restante, la dispersión central se superpone con la secuencial. La evidencia permite distinguir entre una mejora de mediana y una ventaja que satisface la regla práctica de estabilidad elegida. Tampoco establece que tres procesos siempre terminen antes que dos: en una de las rondas ocurre lo contrario.

Los puntos n=224 y n=256 delimitan una región interesante para investigar, sin garantizar un umbral exacto ni una regla para todos los tamaños mayores. El cruce observado corresponde al kernel, con lanzamiento, generación, distribución inicial, reunión y validación fuera del reloj. La siguiente etapa debe comprobar qué ventaja persiste al incluir el flujo necesario para obtener el resultado completo.

IV

PARTE IV

Flujo de datos y cierre local

¿La mejora persiste cuando se mide el trabajo necesario para obtener el factor completo?

¿Quién genera la matriz?

Pregunta e hipótesis

¿Podemos evitar que el proceso 0 genere y distribuya toda la matriz? Si cada proceso calcula sus columnas directamente, eliminamos el empaquetado central y Scatterv. A cambio, se recalculan los valores simétricos y aparecen accesos por columnas; la mejora debe medirse.

Implementación: exactamente la misma matriz

El generador original visita las parejas i<j en orden. Identificamos la posición de cada pareja en esa secuencia y recuperamos el estado del generador entero sin recorrer los valores anteriores. La suma diagonal conserva el mismo orden que antes. Por ello, el resultado se puede comprobar bit a bit.

$$q(i,j) = i(2n - i - 1)/2 + (j - i - 1), \quad i < j$$

$$a_{ii} = 1 + \sum_{j \neq i} |a_{ij}|$$

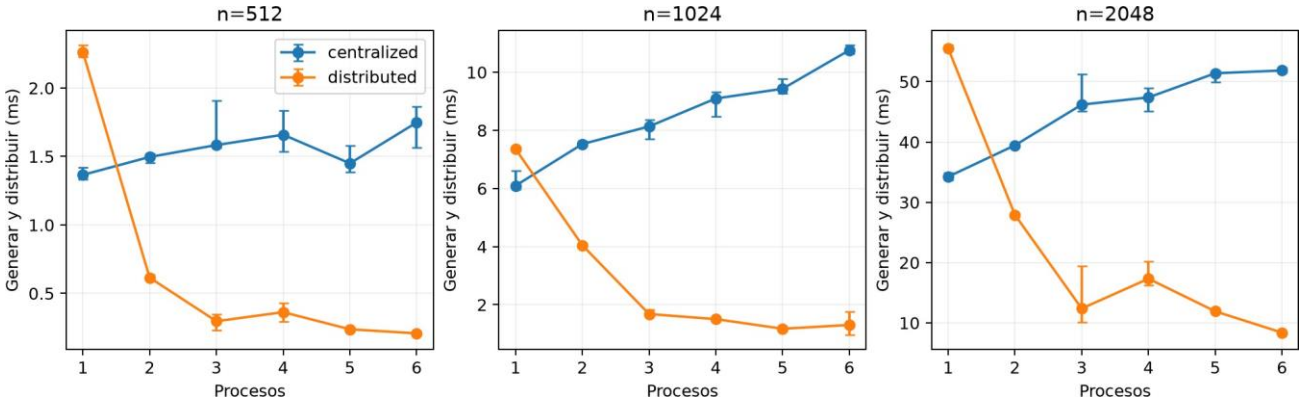
La simetría y la dominancia diagonal estricta con diagonal positiva aseguran que la matriz sea definida positiva. Cada proceso guarda sus columnas cíclicas en el formato que ya utiliza nuestro Cholesky MPI.

Protocolo

WSL, Ryzen 5 2600; 6 núcleos físicos. Tamaños 512, 1024 y 2048; P=1 a 6; semilla 2026; un calentamiento y 6 repeticiones alternadas por método. Total: 216 observaciones. Centralizado: generación en rango 0, empaquetado y Scatterv. Distribuido: generación directa local. Ambos terminan con idéntica matriz distribuida.

El cronómetro excluye reserva, arranque MPI, factorización y validación. Se toma el máximo del tiempo total entre procesos. Las fases no están sincronizadas: sus máximos incluyen esperas distintas y no deben sumarse para reconstruir el total.

Resultados y decisión provisional



n	P	Central (ms)	Distrib. (ms)	Cociente
512	1	1.366	2.262	0.60x
512	6	1.748	0.208	8.40x
1024	1	6.097	7.359	0.83x
1024	6	10.759	1.303	8.26x
2048	1	34.243	55.519	0.62x
2048	6	51.862	8.385	6.18x

Para n=2048 y P=6, el cociente entre medianas de preparación es 6.18. Este cociente compara dos métodos con el mismo P; no es el speedup de Cholesky ni el de la aplicación completa.

Conclusión

La generación directa de columnas reduce el tiempo de preparación con seis procesos en los tres tamaños presentados y conserva exactamente la matriz del método centralizado. Para n=2048 y P=6, la mediana baja de 51,862 ms a 8,385 ms, una mejora de 6,18 entre métodos con igual cantidad de procesos. Evitar la generación central, el empaquetado y Scatterv resulta beneficioso en esas configuraciones.

Con un solo proceso, la generación distribuida es más lenta en los mismos tamaños. Su ventaja depende, por tanto, de poder repartir la preparación y compensar el trabajo adicional del método. La equivalencia bit a bit permite atribuir la comparación a dos formas de producir la misma entrada, sin cambiar el problema numérico.

Incorporamos la generación distribuida al candidato y evaluamos su impacto en el flujo completo. El cociente de preparación mide una fase específica; para establecer el beneficio final es necesario incluir la factorización y la reunión del factor.

Siguiente pregunta

El siguiente banco comprobará cuánto de esta mejora se conserva hasta obtener L completa, incluyendo reserva, preparación, factorización y reunión. El tiempo externo se evaluará por separado.

¿Mejora el tiempo completo?

La generación distribuida redujo el tiempo de preparar la entrada en la etapa anterior. Ahora preguntamos si esa mejora se mantiene al incluir la factorización y devolver el resultado al proceso 0.

Una comparación con el mismo resultado

- Centralizada: rango 0 genera A, empaqueta y distribuye columnas.
- Distribuida: cada proceso genera directamente sus columnas de la misma A.

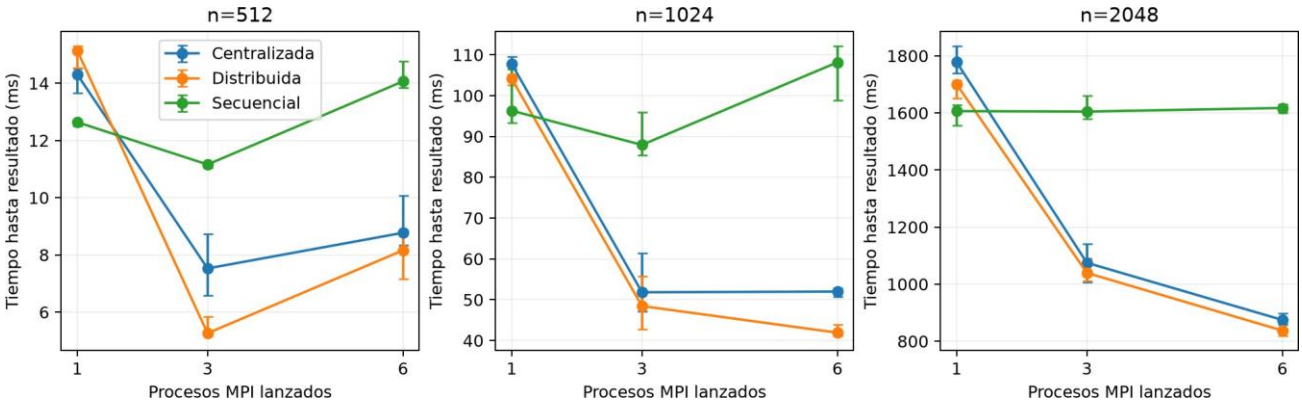
Ambas ejecutan right-looking MPI cíclico con vector local y reúnen L completa en rango 0. La referencia secuencial genera A y aplica right-looking con vector en rango 0.

Qué llamamos total en este experimento

$$T = T_{reserva} + T_{generacion} + T_{distribucion} + T_{factorizacion} + T_{recoleccion}$$

Medimos directamente el intervalo desde el comienzo de las reservas hasta que L queda completa en rango 0, con MPI ya iniciado. Las fases se registran consecutivamente en ese mismo proceso y suman su total por ejecución. También guardamos el máximo de las duraciones locales como control.
Quedan fuera el lanzamiento y cierre de MPI, las referencias de prueba, la validación, la liberación de memoria y escribir resultados. Es el flujo de procesamiento, no el tiempo externo del comando mpirun.

Resultados del flujo integrado



n	P	Central (ms)	Distrib. (ms)	Secuencial (ms)
512	3	7.53	5.26	11.16
512	6	8.77	8.16	14.06
1024	3	51.82	48.44	87.92
1024	6	51.98	41.91	108.15
2048	3	1074.82	1039.34	1604.65
2048	6	874.60	837.39	1617.31

Para n=2048 y P=6: cambiar la preparación centralizada por distribuida da un cociente de 1.04x y una reducción de 4.3 % del tiempo mediano. Respecto del secuencial medido en esa configuración, el método distribuido alcanza 1.93x.

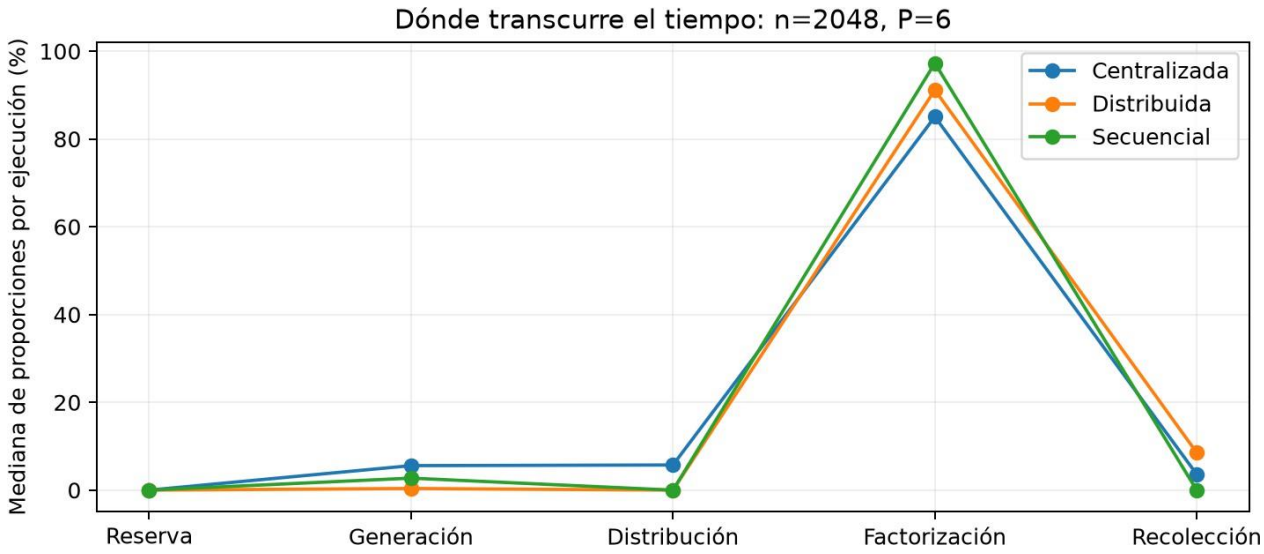
Lo que esta comparación permite afirmar

La generación distribuida mantiene una ventaja de mediana en el flujo medido hasta reunir L completa en rango 0, aunque esa ventaja es mucho menor que la obtenida al medir únicamente la preparación. Para n=2048 y P=6, el tiempo pasa de 874,60 ms a 837,39 ms: una reducción aproximada del 4,3 %. Frente al secuencial contemporáneo de esa configuración, el método distribuido alcanza una aceleración de 1,93.

El resultado identifica dónde concentrar la siguiente mejora. En el flujo distribuido de ese caso, las medianas de las fracciones por ejecución sitúan a la factorización en el 91,1 % del tiempo y a la recolección en el 8,5 %. Reducir una fase preparatoria ya pequeña tiene un efecto limitado sobre el conjunto; el kernel y la salida adquieren mayor prioridad.

La comparación incluye resultados equivalentes y matrices idénticas. Las diferencias observadas dentro de la factorización pueden incorporar efectos del estado de memoria y de la llegada de los procesos, aun usando el mismo algoritmo. El reloj comienza en las reservas y termina con L reunida, con MPI ya iniciado; la ganancia del comando externo queda por evaluar.

¿Dónde conviene trabajar ahora?



En el flujo distribuido de n=2048, P=6, la factorización representa una mediana del 91.1 % del tiempo observado y la recolección del 8.5 %. Estas proporciones sirven para priorizar el próximo experimento local; no predicen el comportamiento de la red del clúster.

¿Agrupar columnas ayuda?

La etapa anterior mostró que la factorización concentra gran parte del tiempo. Estudiamos ahora si cambiar la propiedad de las columnas mejora el flujo completo. Se mantiene right-looking escalar: cada columna se termina y difunde antes de continuar.

Hipótesis

Un bloque de b columnas consecutivas se asigna por turnos a cada proceso. Agrupar columnas puede modificar la localidad y las esperas, pero también el reparto del trabajo. Como las columnas locales ya estaban empaquetadas por filas, b mayor no garantiza mejor acceso a memoria.

$$\text{propietario}(j) = \lfloor j/b \rfloor \bmod P$$

Con P=3 y b=2: proceso 0 recibe 0,1,6,7; proceso 1 recibe 2,3,8,9; proceso 2 recibe 4,5,10,11. b=1 reproduce los propietarios de la distribución cíclica previa.

Qué cambia y qué se conserva

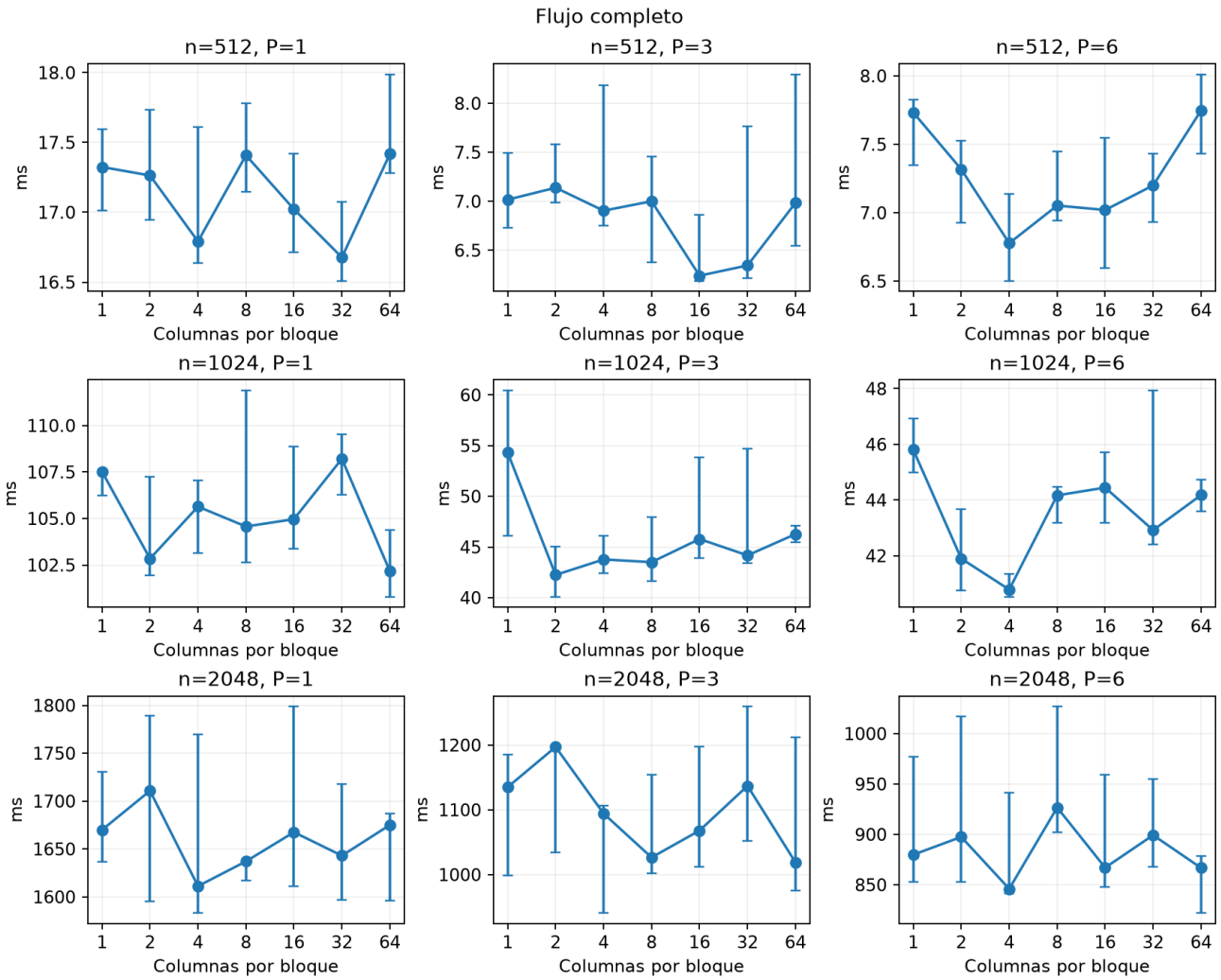
Cambia el mapa global/local y la cantidad de columnas de cada proceso. Se conserva la matriz generada, el kernel de actualización con vector local, n difusiones por factorización y L reunida en rango 0. Todas las variantes usan el mismo código general de índices, incluido b=1. No comparamos contra cifras de campañas anteriores.

Esto es distribución por bloques de columnas. No es Cholesky por bloques con operaciones matriz-matriz ni agrupa las comunicaciones: sigue habiendo una difusión por columna.

Protocolo y reloj

n=512,1024,2048; P=1,3,6; b=1,2,4,8,16,32,64. Un calentamiento por variante y siete repeticiones. El orden rota para que cada bloque ocupe cada posición una vez; no balancea todas las parejas de vecinos. Se rota P entre tamaños.
Total principal: reloj de rango 0 desde reserva y preparación del reparto hasta generación distribuida, factorización y recolección completa.
Excluye lanzamiento/cierre MPI, referencias, validación y liberación. Las fases incluyen esperas. Se compara la mediana de totales medidos directamente.

Tiempo hasta obtener el resultado

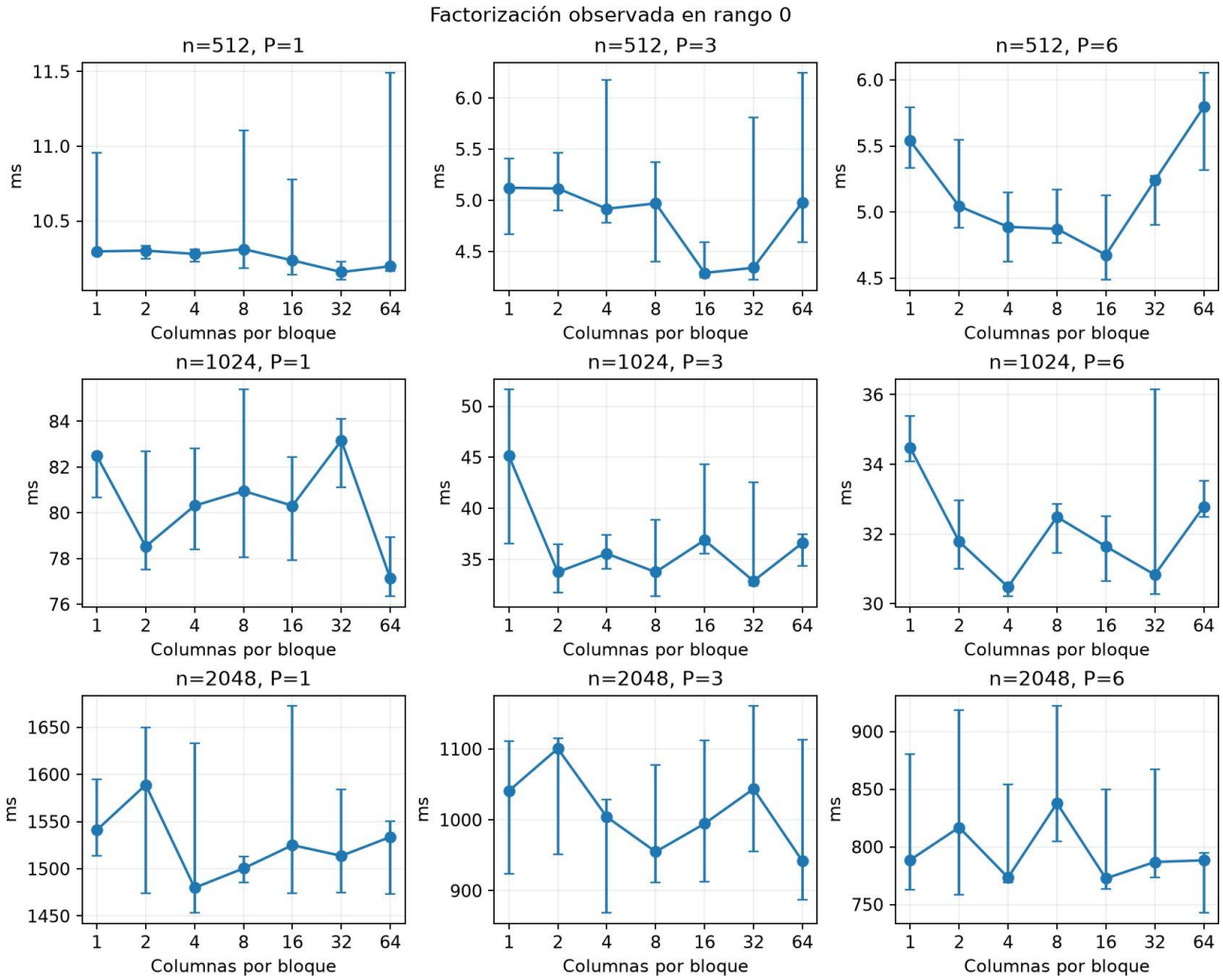


Puntos: medianas; barras: Q1-Q3, no intervalos de confianza. Cada panel tiene su propia escala vertical. Comparamos bloques dentro de cada configuración.

Interpretación

El total contempla los cambios de generación y desempaquetado provocados por el reparto, además del kernel. Una menor mediana aislada es un candidato para repetir, no una demostración de superioridad universal. Elegir el mínimo entre siete alternativas puede favorecer fluctuaciones de esta sesión.

Factorización y reparto del trabajo



Factorización observada por rango 0: incluye sus esperas de comunicación. No es una medición separada de cómputo puro ni del máximo de cada proceso.

$$W_r = \sum_{k \in C_r} k(n-k), \qquad I = \frac{\max_r W_r}{\sum_r W_r / P}$$

Cada columna k recibe k actualizaciones sobre n-k elementos. I=1 indica igualdad en este conteo acumulado; no garantiza balance en cada etapa, ni incluye pivotes, indexación o comunicaciones.

Decisión provisional y siguiente pregunta

n	P	Mejor b observado	b=1 (ms)	Mejor (ms)	Cociente
512	3	16	7.02	6.24	1.124x
512	6	4	7.73	6.78	1.141x
1024	3	2	54.32	42.24	1.286x
1024	6	4	45.81	40.78	1.123x
2048	3	64	1135.03	1018.87	1.114x
2048	6	4	879.80	845.97	1.040x

Estos mínimos son descriptivos y fueron elegidos después de medir. Seleccionamos b=4 para una repetición con P=6, manteniendo b=1 como referencia. No se selecciona un tamaño universal.

Repetición del candidato

n=1024, P=6: b=1 tarda 48.93 ms y b=4 tarda 44.02 ms; cociente 1.112x. Son nuevas ejecuciones, con siete repeticiones por bloque, en la misma sesión local.
n=2048, P=6: b=1 tarda 931.67 ms y b=4 tarda 929.70 ms; cociente 1.002x. Son nuevas ejecuciones, con siete repeticiones por bloque, en la misma sesión local.

La repetición agrega 98 observaciones. Para n = 1024, b = 4 conserva una reducción cercana al 10 %; para n = 2048, la diferencia cae a aproximadamente 0,2 %. La conclusión siguiente integra la exploración y esta repetición.

Conclusión

Cambiar el tamaño de bloque de distribución produce mínimos distintos según el tamaño de matriz y la cantidad de procesos. En la exploración inicial, el mejor b observado va de 2 a 64; no aparece una elección universal. Para n=1024 y P=6, b=4 reduce la mediana de 45,81 ms a 40,78 ms respecto de b=1, y en una repetición dentro de la misma sesión mantiene una reducción cercana al 10 %, de 48,93 ms a 44,02 ms.

La ventaja es menos consistente en n=2048 y P=6: el cociente inicial de 1,040 cae a 1,002 al repetir. Esa reducción de aproximadamente 0,2 % no respalda adoptar b=4 de forma general. La selección del mínimo entre siete alternativas sirve para proponer candidatos que luego deben confirmarse, porque también puede favorecer fluctuaciones de la sesión.

Conservamos b=1 como referencia y b=4 como candidato específico para n=1024, P=6. Esta prueba cambia la propiedad de grupos de columnas y mantiene una difusión por columna. Sus resultados evalúan el reparto y sus efectos sobre el flujo; no miden una factorización por bloques con operaciones matriz-matriz ni una reducción en la cantidad de colectivas.

Cerrar decisiones locales

¿Se mantiene la mejora de cuatro columnas?

b=4 pareció prometedor para n=1024 y P=6. Lo repetimos en esta nueva sesión con b=1 y b=4, ocho repeticiones por variante, orden alternado y el mismo resultado reunido en rango 0. El total contiene reserva, generación distribuida, kernel y recolección; MPI ya está iniciado.

n	P	b=1 total (ms)	b=4 total (ms)
1024	6	42.34	43.36
2048	6	767.99	798.87
3072	6	4030.13	4038.46

Conclusión

La nueva sesión no confirma la ventaja de b=4 obtenida anteriormente. Para n=1024 y P=6, la mediana total pasa de 42,34 ms con b=1 a 43,36 ms con b=4; para n=2048, pasa de 767,99 ms a 798,87 ms. En n=3072, las medianas de 4030,13 ms y 4038,46 ms son muy próximas y tampoco favorecen a b=4.

La evidencia acumulada muestra que la mejora inicial depende de las condiciones de medición y carece de confirmación entre estas sesiones. Por ello, mantenemos b=1 como referencia principal y dejamos b=4 disponible para futuras comparaciones, sin presentarlo como una optimización consolidada. Repetir el candidato permitió evitar una decisión general basada en un mínimo favorable de una sola campaña.

¿Qué aporta LAPACK a la comparación?

LAPACK proporciona rutinas de álgebra lineal. DPOTRF calcula Cholesky y organiza operaciones por bloques utilizando BLAS de nivel 3. LAPACKE es su interfaz C; OpenBLAS suministra rutinas optimizadas para el procesador. Esta referencia combina algoritmo e implementación optimizada: no aísla cuánto aporta cada técnica.

Comparación controlada desde C

Nuestro right-looking con vector y LAPACK se ejecutan en el mismo programa, sin MPI, ligados a CPU 0 en esta PC. OpenBLAS se configura en un hilo y se comprueba con su API. Misma semilla 2026, precisión double, generador y política de reserva. Ambos reservan un vector n; LAPACK no necesita usarlo. Un calentamiento y seis repeticiones alternadas para n=512,1024,2048; dos para n=3072.

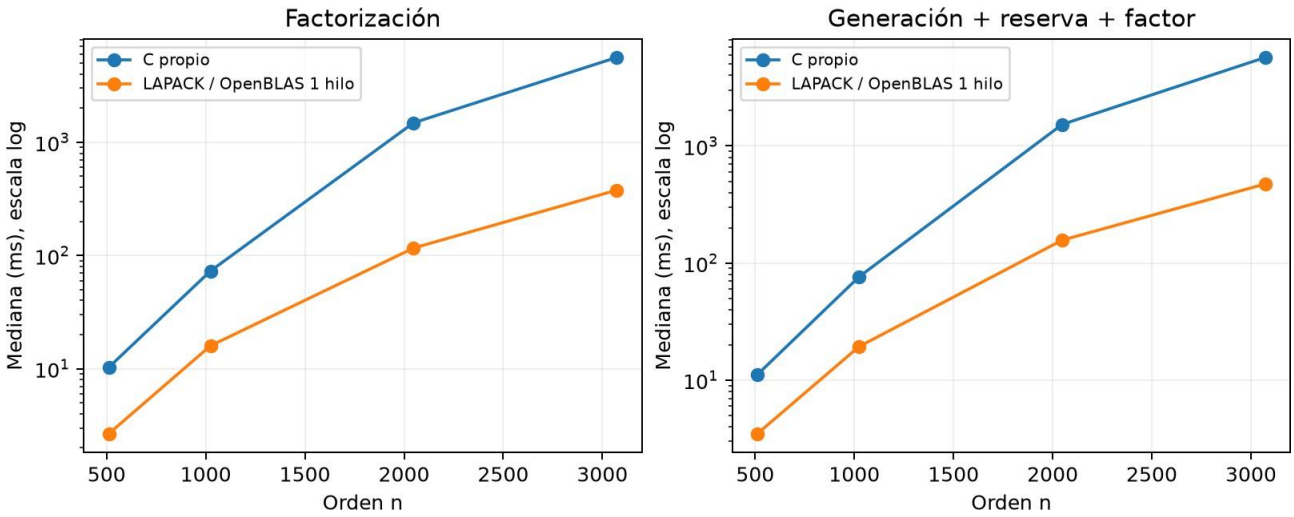
Tiempo de factor: llamada de factorización. Total: reserva + generación + factor.
Referencias, validación y liberación están fuera.
No hay conversión de almacenamiento: A simétrica por filas se interpreta por columnas, DPOTRF produce U y el mismo arreglo representa L=U transpuesta por filas. Un caso con factor conocido comprueba esa interpretación.

Usamos LAPACKE_dpotrf_work para evitar una transposición de la interfaz row-major y su barrido opcional de NaN. El dominio medido es entrada finita SPD generada; validamos resultados por separado. Los contratos de error de las implementaciones no son idénticos.

Bibliotecas y fuentes consultadas

LAPACKE 3.12.1; OpenBLAS 0.3.32, backend Zen, un hilo comprobado. Los logs incluyen dependencias dinámicas y configuración. La optimización interna de OpenBLAS no usa necesariamente las mismas opciones que nuestro código compilado con -O3.

Resultados: factor y tiempo completo



Medianas; escala vertical logarítmica. n=3072 es exploratorio con dos repeticiones por método. No se mezclan tiempos de MPI con este experimento secuencial.

n	C factor ms	LAPACK factor ms	C total ms	LAPACK total ms
512	10.33	2.68	11.19	3.50
1024	73.03	15.96	76.37	19.37
2048	1484.35	116.30	1527.24	156.85
3072	5603.73	377.22	5701.11	473.59

Para n=2048, el cociente C/LAPACK es 12.76x en factor y 9.74x en total. Es una comparación de implementaciones con un hilo, no speedup MPI ni prueba de que un algoritmo sea siempre superior.

Qué decidimos con estos datos

LAPACK/OpenBLAS con un hilo supera ampliamente al kernel secuencial propio en los tamaños ensayados. Para n=2048, la factorización tarda 116,30 ms frente a 1484,35 ms del código propio, con cociente de 12,76. Al incluir reserva y generación, los tiempos son 156,85 ms y 1527,24 ms, y el cociente disminuye a 9,74.

La diferencia demuestra que, después de mejorar el acceso con un vector, todavía queda un margen considerable de rendimiento local. También muestra que optimizar la factorización aumenta la importancia relativa de las fases que permanecen en el flujo. LAPACK aporta una referencia exigente para evaluar nuevas versiones del kernel y para estudiar una implementación realmente bloqueada con BLAS de nivel 3.

La comparación no separa el aporte de las técnicas internas de la biblioteca. N = 3072, con dos repeticiones por método, es exploratorio. Adoptamos LAPACK como referencia para mejorar el rendimiento local.

Speedup del candidato

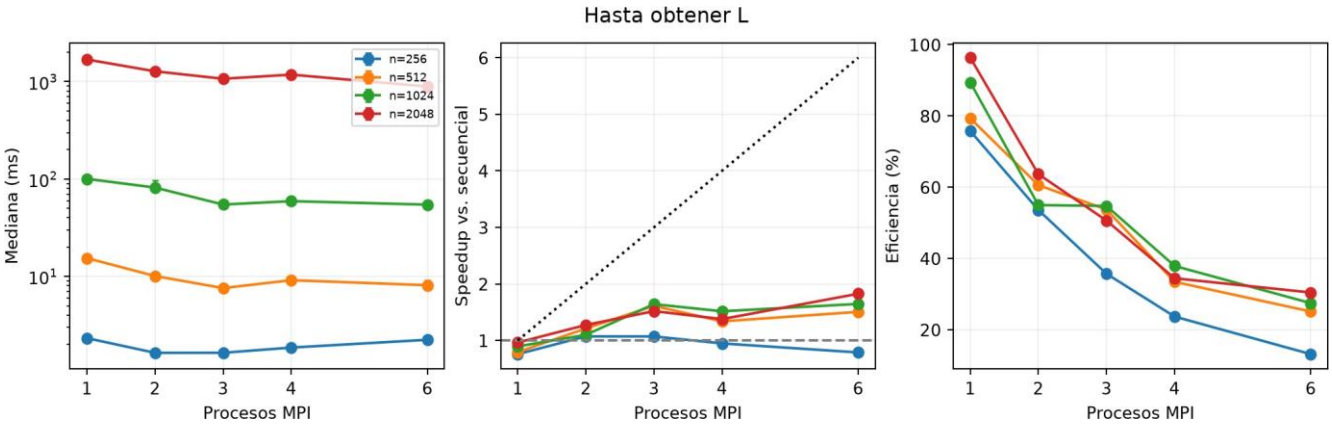
¿Cuánto ganamos hasta obtener L?

Medimos el programa elegido frente al secuencial autónomo contemporáneo, sin reutilizar tiempos anteriores. El objetivo es observar aceleración útil y eficiencia, no solamente comparar dos optimizaciones con igual P.

$$S_{app}(n,P) = \frac{\text{mediana}(T_{app,seq})}{\text{mediana}(T_{app,MPI,P})}, \qquad E_{app} = S_{app}/P$$

Protocolo: N=256,512,1024,2048; MPI P=1,2,3,4,6 y secuencial. Siete repeticiones por configuración, más una descartada; 168 observaciones medidas y 24 de preparación. Cada observación es un lanzamiento nuevo. Rotamos el orden de los seis métodos entre rondas; los tamaños siguen un orden fijo. No es una aleatorización completa.

WSL local, Ryzen 5 2600, seis núcleos físicos y doce procesadores lógicos. MPI solicita un proceso por núcleo con afinidad; secuencial usa taskset CPU 0. GCC -O3, double, sin BLAS. Una sola sesión: no inferimos estabilidad entre días ni entre nodos.



Puntos: medianas. Barras en tiempos: Q1-Q3 inclusivos. Los cuartiles describen dispersión, no intervalos de confianza. Las curvas de speedup son cocientes de medianas, sin incertidumbre inferencial calculada.

Lectura de la figura

S mayor que 1 significa que MPI reduce el tiempo frente a nuestro secuencial; E muestra la fracción del ideal lineal respecto de esa referencia. MPI con P=1 no tiene por qué coincidir con el secuencial: cambia almacenamiento y agrega coordinación y reunión.

Resultados y decisión local

N	P	MPI ms	S app	E app
256	1	2.32	0.76	75.7%
256	2	1.64	1.07	53.6%
256	3	1.64	1.07	35.7%
256	4	1.86	0.95	23.7%
256	6	2.23	0.79	13.1%
512	1	15.41	0.79	79.3%
512	2	10.09	1.21	60.6%
512	3	7.58	1.61	53.7%
512	4	9.14	1.34	33.4%
512	6	8.12	1.51	25.1%
1024	1	100.43	0.89	89.4%
1024	2	81.68	1.10	55.0%
1024	3	54.69	1.64	54.7%
1024	4	59.25	1.52	37.9%
1024	6	54.54	1.65	27.4%
2048	1	1686.75	0.96	96.3%
2048	2	1276.59	1.27	63.6%
2048	3	1070.06	1.52	50.6%
2048	4	1180.06	1.38	34.4%
2048	6	890.19	1.83	30.4%

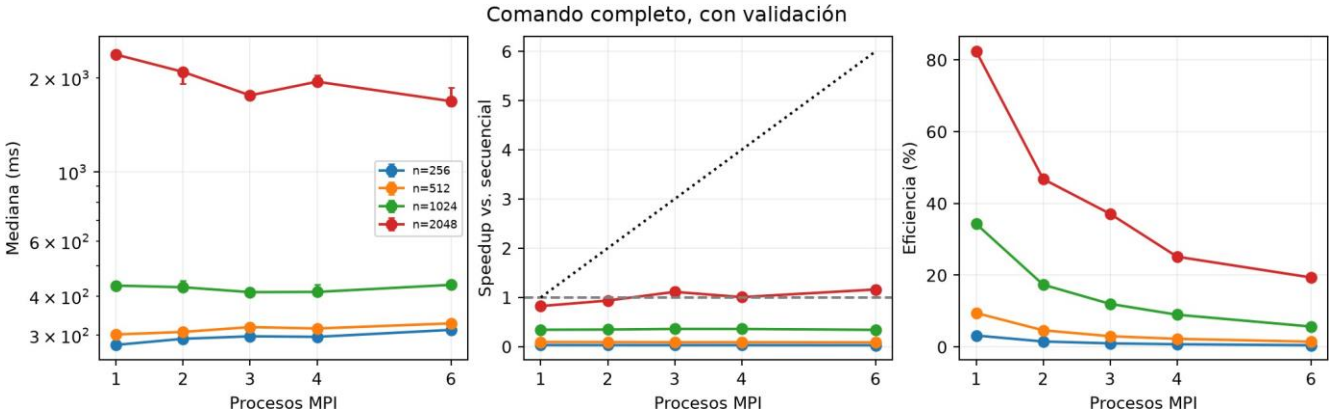
Conclusión

MPI reduce el tiempo hasta L en varias configuraciones. Para N = 2048, seis procesos alcanzan la menor mediana ensayada, 890,19 ms, con aceleración de 1,83 y eficiencia relativa del 30,4 %. En N = 256, la ventaja de 1,07 con dos o tres procesos es pequeña.

En N = 1024, tres y seis procesos tardan 54,69 ms y 54,54 ms: duplicar los recursos apenas cambia la latencia y reduce la eficiencia de 54,7 % a 27,4 %. Tres procesos son una elección razonable si se busca ahorrar recursos. La conveniencia final debe comprobarse también con el comando completo.

El comando completo

$$S_{ext}(n, P) = \frac{\text{mediana}(T_{ext,seq})}{\text{mediana}(T_{ext,MPI,P})}$$



N	Seq ext ms	MPI P6 ext ms	S ext P6
256	8.89	312.08	0.03
512	28.53	327.38	0.09
1024	148.30	434.99	0.34
2048	1957.48	1685.41	1.16

La validación se realiza en rango 0 mientras los demás procesos esperan el estado final. Este trabajo queda fuera de application_s y dentro del reloj externo; sus costos pueden variar entre ejecuciones.

Qué queda decidido

Con seis procesos, el comando MPI tarda más que el secuencial en N = 256, 512 y 1024. En N = 2048 alcanza 1685,41 ms frente a 1957,48 ms, con aceleración externa de 1,16: es el primer tamaño favorable de esta grilla local para P = 6.

La ganancia hasta L puede quedar absorbida por el lanzamiento, la validación, las esperas y el cierre. Para reducir la espera del usuario, la elección debe apoyarse en el reloj externo. Conservamos right-looking MPI cíclico con b = 1 y ajustamos la cantidad de procesos al tamaño y al alcance de la ejecución.

V

PARTE V

Campañas en el clúster y comparación final

¿Qué mejora sobrevive en el clúster y cuánto rendimiento local queda sin explotar?

Campaña formal de cruce

Pregunta y resultado

¿En qué tamaños ensayados MPI reduce claramente el tiempo hasta disponer de L frente al secuencial propio?

Protocolo

N=128,160,192,224,256; SEQ y MPI P=1,2,3,4,6. Una ronda descartada y siete medidas. Orden de métodos rotado; tamaños en orden creciente. Afinidad prevista: SEQ con taskset CPU0; MPI --bind-to core --map-by core. El entorno conserva un diagnóstico de binding P6 en el socket 0, no un registro independiente de cada lanzamiento.

AMD EPYC 7B12, GCC 13.3, Open MPI 4.1.6, cuota de seis CPUs equivalentes y 24 GiB. Fuente de entorno fechada 2026-09-28T16:06:20Z. No se interpreta la cuota como seis cores exclusivos ni como acceso a varios nodos

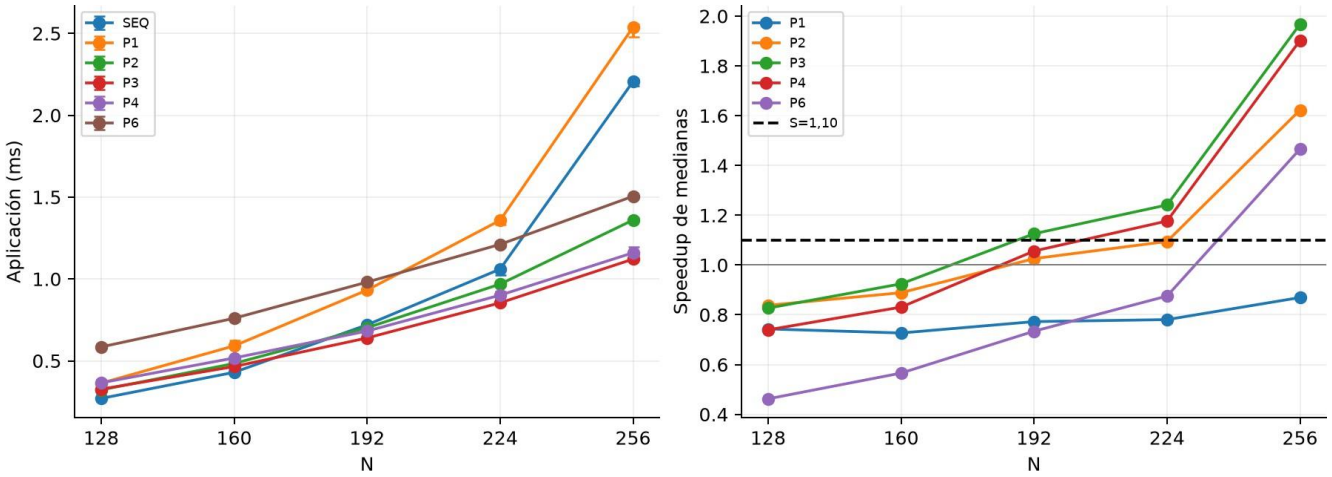


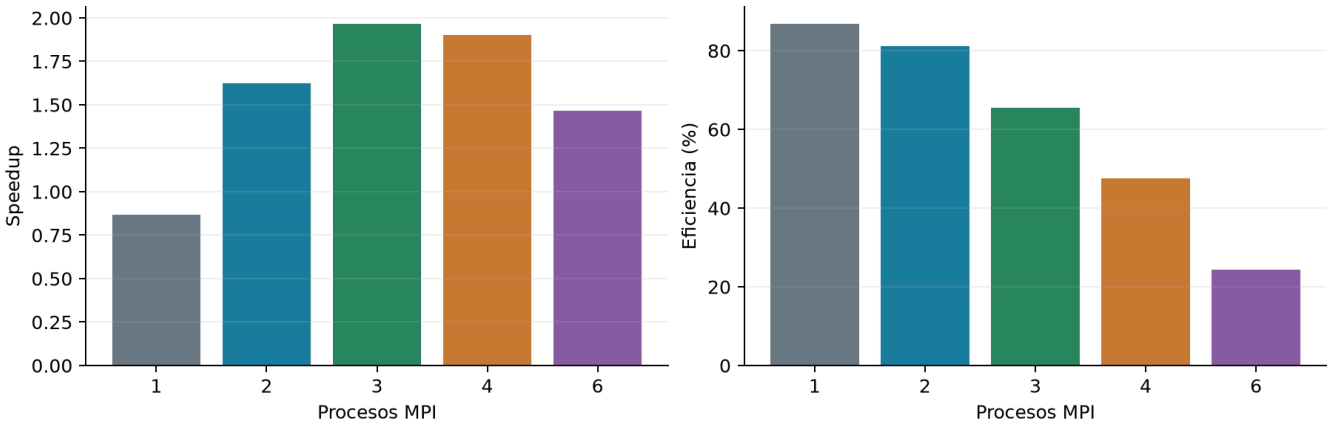
Figura 1. Medianas y barras Q1-Q3 inclusivos en tiempos, siete observaciones por punto. Los speedups son cocientes de medianas, sin intervalos de confianza. Las líneas unen únicamente los puntos medidos. Warmups y pilotos quedan excluidos.

S=mediana(T_SEQ)/mediana(T_MPI). E=S/P. Mejora clara descriptiva si S>1,10 y Q3_MPI<Q1_SEQ, ambas desigualdades estrictas. Cuartiles por interpolación inclusiva. No es una prueba formal de significación y no demuestra monotonicidad para tamaños no ensayados.

N	P1	P2	P3	P4	P6
128	No	No	No	No	No
160	No	No	No	No	No
192	No	No	Sí	No	No
224	No	No	Sí	Sí	No
256	No	Sí	Sí	Sí	Sí

Procesos	Primer N ensayado con mejora clara
1	Ninguno en la grilla
2	256
3	192
4	224
6	256

N=256: rapidez y eficiencia son preguntas distintas



Método	Q1 (ms)	Mediana (ms)	Q3 (ms)	S	E (%)
SEQ	2.177	2.206	2.219	-	-
P1	2.480	2.538	2.556	0.87	86.9
P2	1.352	1.360	1.374	1.62	81.1
P3	1.112	1.121	1.147	1.97	65.6
P4	1.154	1.160	1.194	1.90	47.6
P6	1.493	1.505	1.508	1.47	24.4

P=3 tiene la menor mediana MPI observada. P=2 obtiene la mayor eficiencia entre las configuraciones que aceleran (P>=2). P=1 da 86,9% de eficiencia numérica, pero S=0,87: es más lento que SEQ. P=6 mejora frente a SEQ con eficiencia 24,4%. Son descripciones de esta campaña; no prueban que P=3 sea siempre óptimo ni que su diferencia frente a P=4 sea estadísticamente concluyente.

Conclusión parcial

Aumentar procesos no asegura menor tiempo. En matrices pequeñas los costos adicionales compiten con el trabajo que se reparte; una configuración intermedia puede terminar antes que utilizar toda la cuota disponible.

Cómo se amortizan los costos adicionales

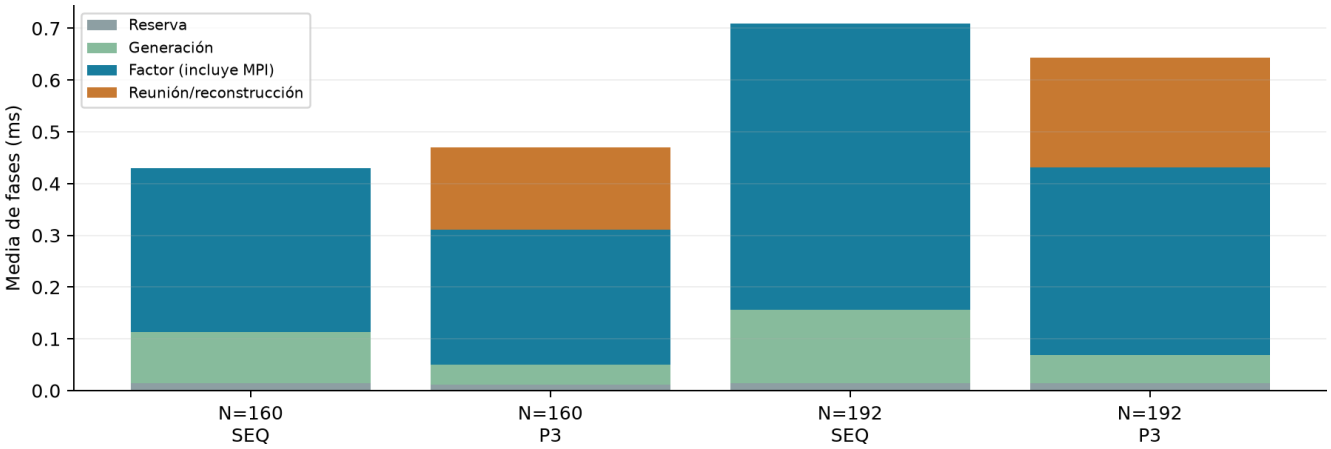


Figura 3. Medias de fases en las siete repeticiones, desde el reloj del rango 0. Aquí se usan medias para que las barras apiladas sumen la media total. El criterio principal sigue usando medianas del total medido, no la suma de medianas.

N / método	Generación ms	Factor ms	Reunión ms	Total ms
160 / SEQ	0.0989	0.3182	0.0000	0.4298
160 / P3	0.0382	0.2607	0.1544	0.4651
192 / SEQ	0.1425	0.5638	0.0000	0.7200
192 / P3	0.0535	0.3599	0.2088	0.6396

Tabla: medianas de cada campo por separado. En N=160/P3, generación y factor disminuyen respecto de SEQ, pero el costo de reunión/reconstrucción impide mejorar el total. En N=192/P3 el ahorro compensa esos costos y se cumplen las dos condiciones de mejora. Esto describe los intervalos medidos; no aísla causas de hardware.

Conclusión y siguiente pregunta experimental

Conclusión de la campaña

Bajo el criterio descriptivo prefijado, la primera mejora clara del tiempo hasta disponer de L aparece en N=192 con tres procesos dentro de la grilla ensayada. Para cuatro procesos aparece en N=224, y para dos y seis en N=256. MPI con un proceso no cumple el criterio en los tamaños probados. El cruce depende de la configuración y del entorno, por lo que el resultado local de n=256, medido con otro alcance temporal, no debe trasladarse directamente a esta campaña.

La comparación de fases ayuda a entender la transición: en N=160, reducir generación y factorización no alcanza para compensar la reunión y reconstrucción; en N=192, el ahorro permite cumplir las dos condiciones de mejora. En N=256, tres procesos alcanzan la menor mediana, 1,121 ms, y aceleración de 1,97. Dos procesos obtienen la mayor eficiencia entre las configuraciones que aceleran, 81,1 %, mientras seis presentan una eficiencia del 24,4 %.

Estos resultados respaldan usar una cantidad intermedia de procesos para matrices pequeñas. La conclusión corresponde a esta sesión, su reloj de aplicación y un criterio de aceleración y separación de cuartiles; no establece un umbral exacto ni un óptimo universal. La cuota de seis CPUs equivalentes tampoco representa seis núcleos exclusivos ni una ejecución multinodo.

Siguiente etapa

Una confirmación selectiva en otra sesión permitiría comprobar la estabilidad alrededor del primer cruce. La etapa siguiente amplía los tamaños y evalúa cuánto de la aceleración hasta L se conserva al medir el comando completo.

Escalabilidad y tiempo completo en el clúster

La campaña estudia N = 512, 1024, 2048 y 4096, con la implementación MPI de uno a seis procesos y una referencia secuencial contemporánea para cada tamaño. La pregunta es cómo cambia el tiempo al repartir más trabajo y qué parte de esa ventaja llega a la ejecución completa.

Se presentan dos relojes. El de aplicación termina cuando L queda disponible e incluye la preparación y la reconstrucción correspondientes; la validación permanece fuera. El externo mide el comando completo y también recoge el lanzamiento, la validación, las esperas y el cierre.

Cada aceleración utiliza la referencia secuencial y el reloj de su propia comparación. La eficiencia S/P expresa rendimiento relativo a esa referencia y no utilización de CPU. La elección de procesos se interpreta junto con el tamaño y el objetivo de la ejecución.

Ganancia hasta obtener L

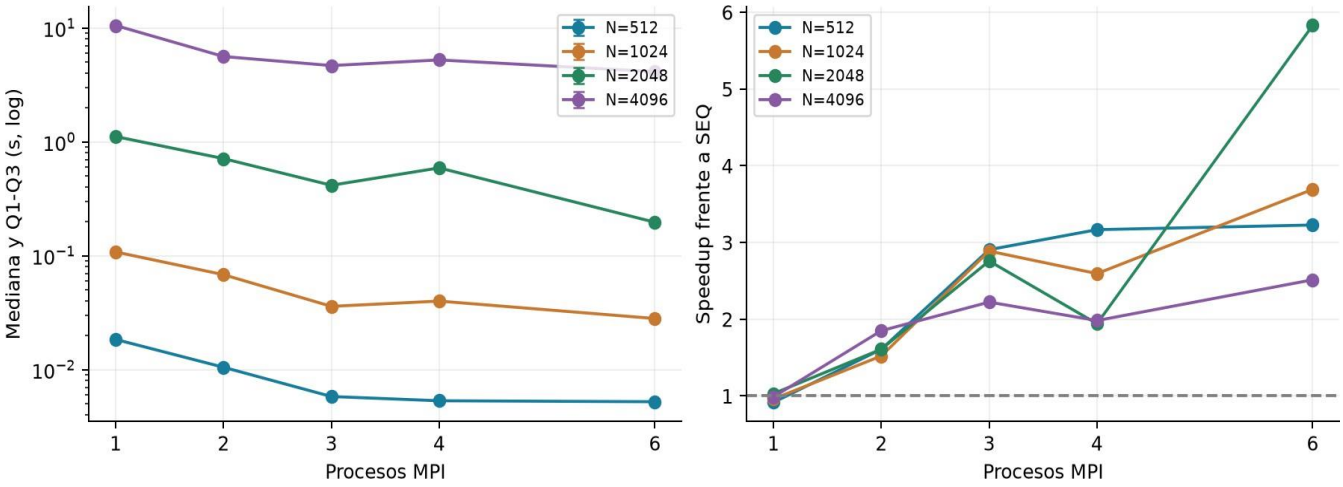


Figura 1. Tiempo de aplicación MPI y cociente mediana(SEQ)/mediana(MPI). Cada N usa su propio secuencial contemporáneo. P1 es la ruta MPI, no una copia del secuencial autónomo.

N	SEQ s	P mínimo	MPI s	S	E %
512	0.016851	6	0.005221	3.23	53.8
1024	0.103648	6	0.028074	3.69	61.5
2048	1.146293	6	0.196693	5.83	97.1
4096	10.391210	6	4.133791	2.51	41.9

Interpretación

Seis procesos alcanzan la menor mediana de aplicación en los cuatro tamaños ensayados. Las aceleraciones frente al secuencial contemporáneo son 3,23 para N = 512, 3,69 para N = 1024, 5,83 para N = 2048 y 2,51 para N = 4096. La ventaja persiste hasta obtener L, pero cambia con el tamaño.

La eficiencia relativa de 97,1 % en N = 2048 expresa cercanía al ideal respecto de esa referencia; no mide utilización de CPU ni identifica un mecanismo causal. En N = 4096 cae a 41,9 %. Además, aumentar P no mejora siempre el tiempo para un tamaño fijo. La configuración debe elegirse mediante tiempos medidos y comprobarse después con el reloj externo.

Pregunta siguiente

¿El mínimo del tiempo interno sigue siendo útil para quien espera que termine el comando? La validación secuencial y el costo de lanzar MPI pueden cambiar la respuesta, especialmente en los tamaños menores.

Cuánto sobrevive en el reloj externo

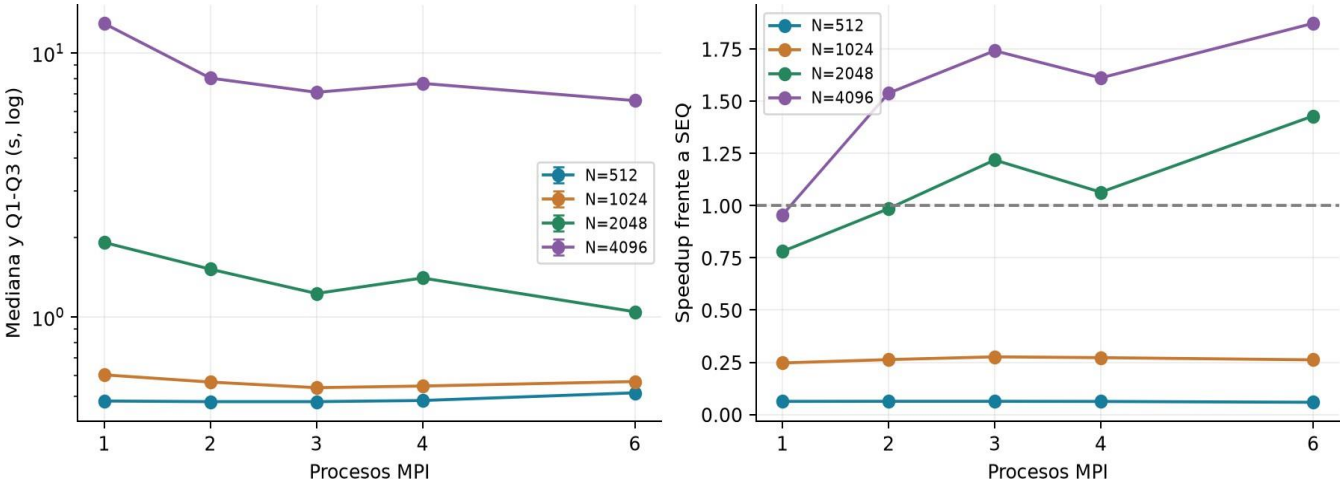


Figura 2. Tiempo del comando y speedup externo. Las barras representan Q1-Q3 del tiempo; no se transfieren las ganancias del reloj interno al externo.

N	SEQ ext s	P mínimo	MPI ext s	S externo
512	0.0302	3	0.4774	0.06
1024	0.1489	3	0.5392	0.28
2048	1.4950	6	1.0459	1.43
4096	12.3637	6	6.6011	1.87

Una comparación explícita con P = 6

N	S aplicación	S externo	E externa %
512	3.23	0.06	1.0
1024	3.69	0.26	4.4
2048	5.83	1.43	23.8
4096	2.51	1.87	31.2

En N = 512 y N = 1024, incluso la menor mediana externa MPI supera a la secuencial. En N = 2048 y N = 4096, seis procesos reducen el comando completo, con aceleraciones de 1,43 y 1,87. N = 2048 es el primer tamaño favorable de esta grilla para el reloj externo; el cruce de aplicación en N = 192 corresponde a otro alcance de medición.

Costos del flujo y conclusión de escalabilidad

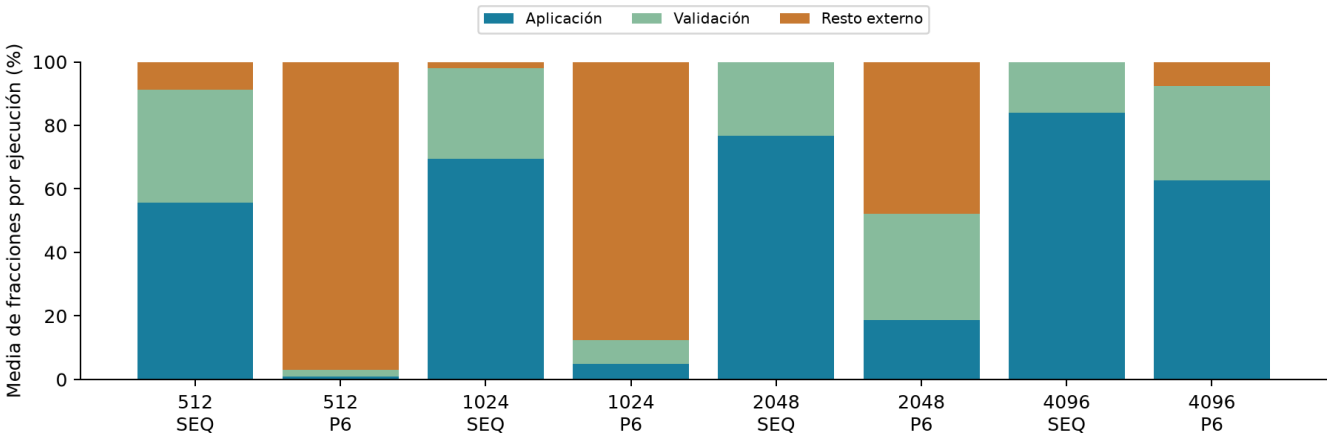


Figura 3. Media de las fracciones calculadas por lanzamiento; cada barra suma 100%. El resto externo combina inicialización, coordinación, cierre y otros costos. No es comunicación pura ni se obtiene restando medianas.

Respuesta experimental

El beneficio del comando completo es menor que el obtenido hasta disponer de L. Con P = 6, la aceleración pasa de 5,83 a 1,43 en N = 2048 y de 2,51 a 1,87 en N = 4096. La mayor aceleración interna corresponde al primer tamaño, mientras el mayor cociente externo corresponde al segundo.

La diferencia cambia la decisión práctica: para reducir la espera hasta que termina el comando, hay que considerar también el lanzamiento, la validación, las esperas y el cierre. En esta grilla, MPI aporta un beneficio externo desde el punto ensayado N = 2048; ese resultado no establece un umbral universal.

La descomposición en fracciones por lanzamiento permite priorizar esos costos adicionales. El resto externo combina varios trabajos y no debe interpretarse como comunicación pura ni calcularse mediante restas de medianas. La campaña confirma aceleración de aplicación y beneficio de extremo a extremo en condiciones específicas del entorno medido.

Escalabilidad fuerte N = 8192 y piloto N = 16384

64.11 s → 13.54 s

4.734× de speedup · 14.79% de eficiencia

Resultado formal para N=8192: medianas de siete mediciones por cantidad de procesos, con referencia MPI P=1 y mejor tiempo observado en P=32.

Diseño experimental y entorno

El strong scaling mantiene fijo N y modifica P. Se evalúa la implementación MPI propia con P=1, 2, 4, 8, 16 y 32. La referencia de speedup es el mismo programa MPI con un proceso; no es una comparación contra LAPACK ni contra un ejecutable secuencial distinto.

Serie	Diseño	Uso
Formal N=8192	7 mediciones/P + 1 warmup/P	42 observaciones útiles
Piloto N=8192	1 observación/P	Exploración previa, excluida
Piloto N=16384	1 observación/P	Exploración, sin dispersión

Recurso	Evidencia y alcance
CPU	Cuota 4800000/100000 = 48 CPUs equivalentes; cpuset 0-127. No equivale a 128 cores asignados.
Memoria	206158430208 bytes = 192 GiB por cgroup.
Software	GCC 13.3.0; Open MPI 4.1.6, según entorno.txt.
Afinidad	Control P=32: ranks 0-31 en socket 0, cores 0-31.
Host	jupyter-trigotex-gmail-com---5306eb41. Control de binding en un host visible.

Orden real conservado en mediciones.csv

Repetición	Orden de P
1	1 2 4 8 16 32
2	2 4 8 16 32 1
3	4 8 16 32 1 2
4	8 16 32 1 2 4
5	16 32 1 2 4 8
6	32 1 2 4 8 16
7	32 16 8 4 2 1

La rotación reduce el sesgo de posición, pero no constituye aleatorización.

Resultados formales: N = 8192

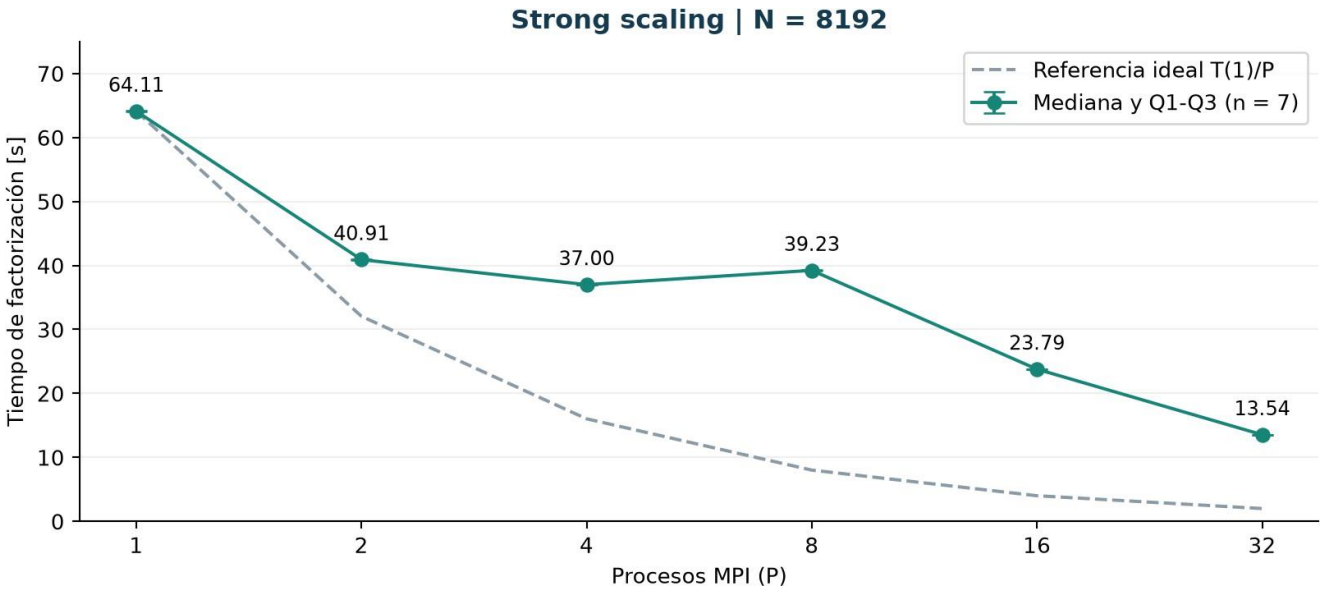


Figura 1. Medianas de siete observaciones; barras Q1-Q3 descriptivas, no intervalos de confianza. Su pequeño tamaño puede hacerlas poco visibles.

P	Mediana [s]	Q1 [s]	Q3 [s]	Speedup	Efic. [%]
1	64.109724	64.085799	64.181409	1.000	100.00
2	40.909027	40.880406	40.924548	1.567	78.36
4	36.998314	36.974880	37.028154	1.733	43.32
8	39.229483	39.141549	39.326518	1.634	20.43
16	23.787586	23.784437	23.792451	2.695	16.84
32	13.543594	13.537468	13.548740	4.734	14.79

P=32 tuvo el menor tiempo mediano entre las seis configuraciones. La reducción absoluta frente a P=1 fue de 50.566130 s por factorización. El ideal T(1)/32 sería 2.003429 s; se utiliza como referencia matemática, no como predicción alcanzable.

Speedup y eficiencia

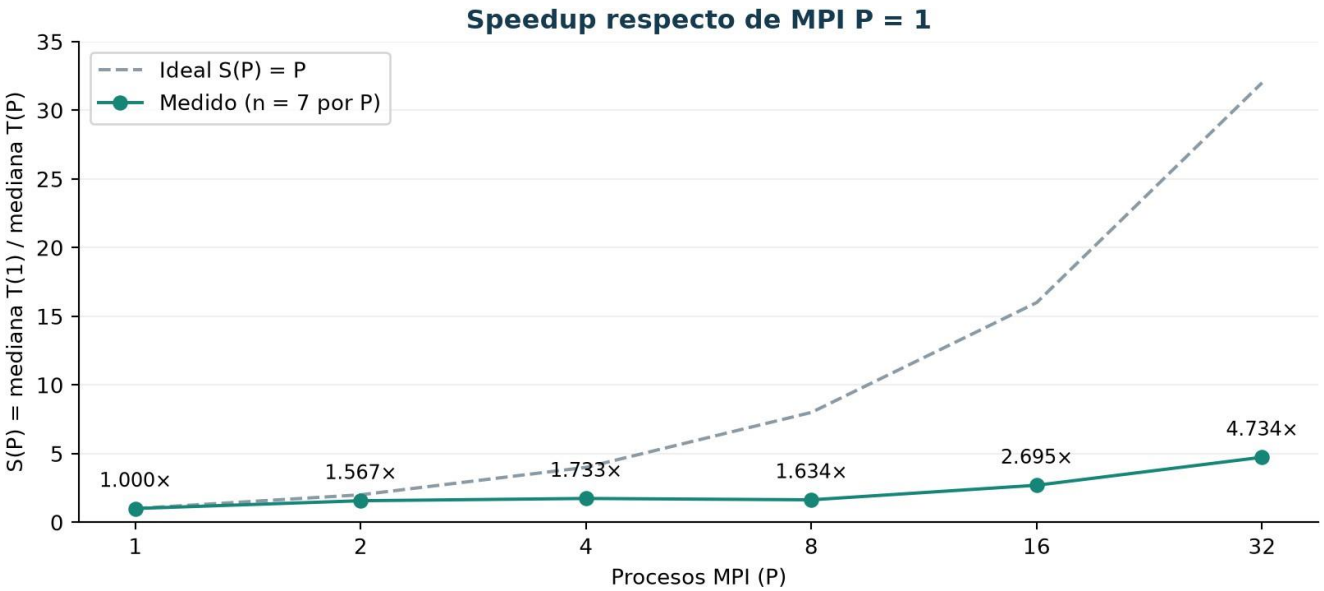


Figura 2. Speedup calculado con las medianas formales y referencia ideal S(P)=P. No se estiman intervalos de confianza para las razones.

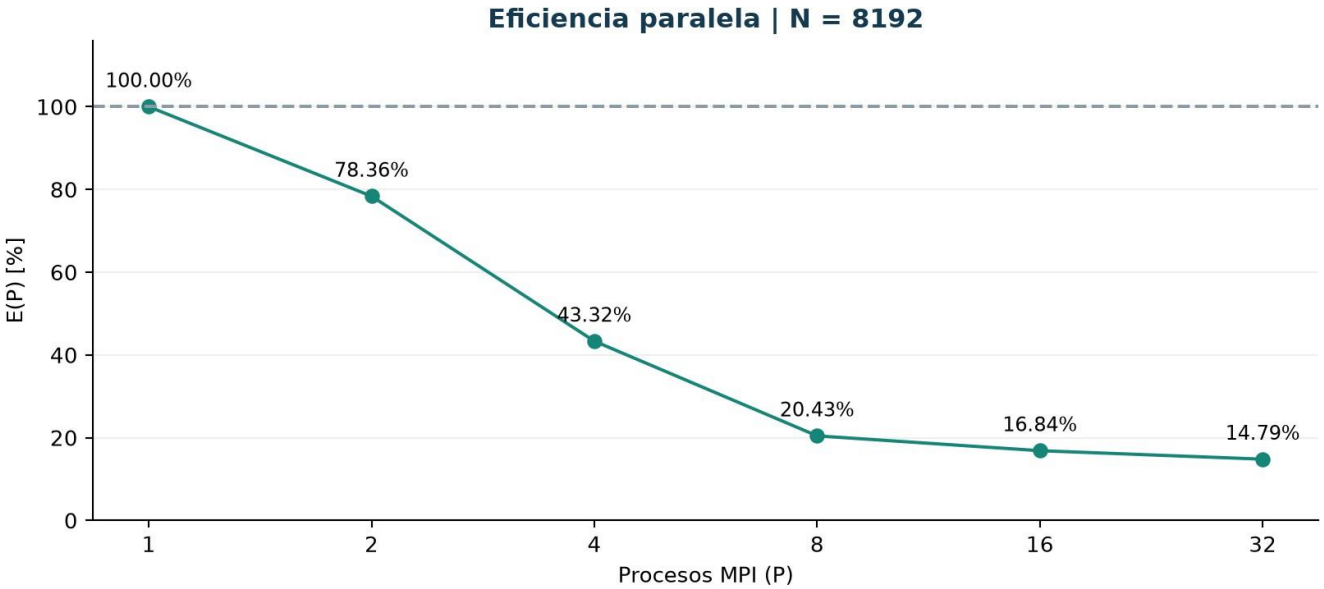


Figura 3. Eficiencia relativa al caso MPI P=1. La eficiencia en P=1 es 100% por definición, no una medición de utilización de CPU.

El speedup alcanza 4.734× con 32 procesos, mientras la eficiencia baja a 14.79%. Menor tiempo de respuesta y mayor aprovechamiento relativo de procesos son criterios distintos. La eficiencia no permite por sí sola separar comunicación, cómputo, memoria y espera.

Dispersión y degradación en P = 8

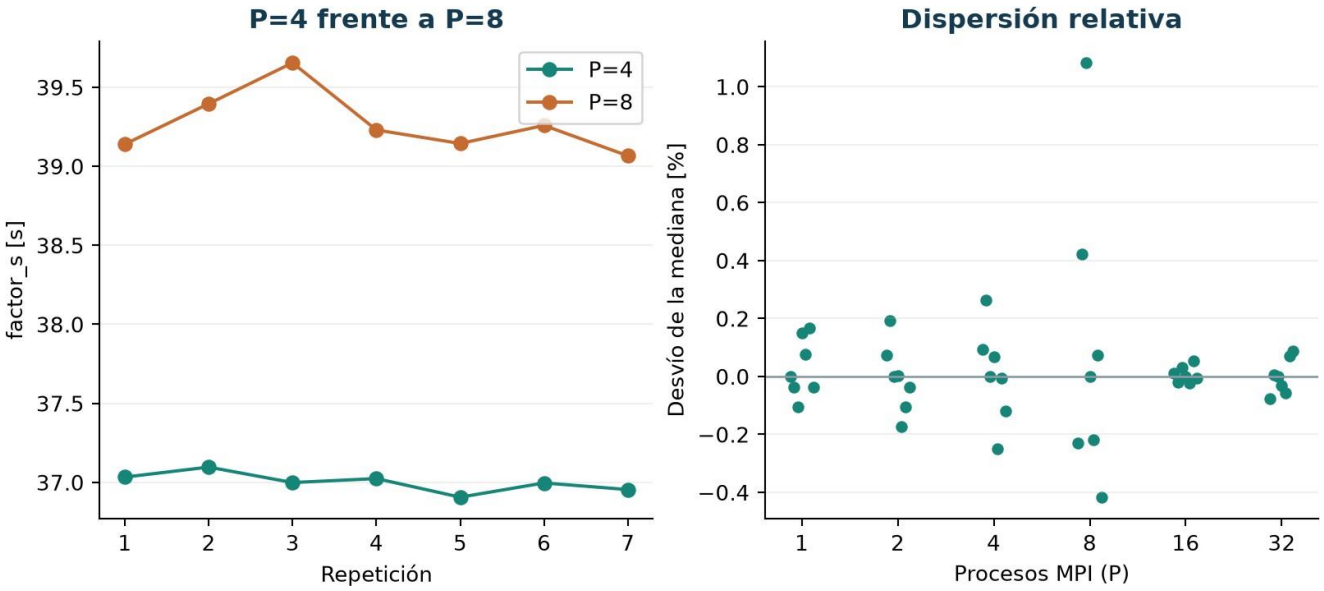


Figura 4. Izquierda: las siete mediciones de P=4 y P=8. Derecha: observaciones relativas a la mediana de cada P; todos los puntos corresponden a datos reales.

P	Media [s]	DE [s]	CV [%]	Mínimo [s]	Máximo [s]
1	64.128979	0.065807	0.1026	64.041956	64.216753
2	40.906374	0.048370	0.1182	40.838299	40.987383
4	37.000883	0.060403	0.1632	36.905717	37.096085
8	39.269368	0.199836	0.5089	39.065890	39.654071
16	23.789107	0.006478	0.0272	23.782221	23.800166
32	13.543490	0.008420	0.0622	13.532982	13.555441

P=8 fue 2.231169 s más lento que P=4 según las medianas, una degradación de **6.03%**. Los Q1-Q3 no se solapan; tampoco los rangos mínimo-máximo de estas dos configuraciones. Las siete comparaciones dentro de cada repetición tienen el mismo sentido.

DE: desviación estándar muestral con divisor n-1. CV=100×DE/media. El bajo CV describe repetibilidad dentro de esta sesión; no mide variación entre días o máquinas. La no superposición de IQR es evidencia descriptiva, no una prueba de significación ni un intervalo de confianza.

La campaña no instrumentó colectivas, desbalance, caché o tiempos por rango de cada fase. Atribuir la anomalía a MPI_Bcast, red o memoria sería una hipótesis sin contrastar.

Tiempo de la aplicación

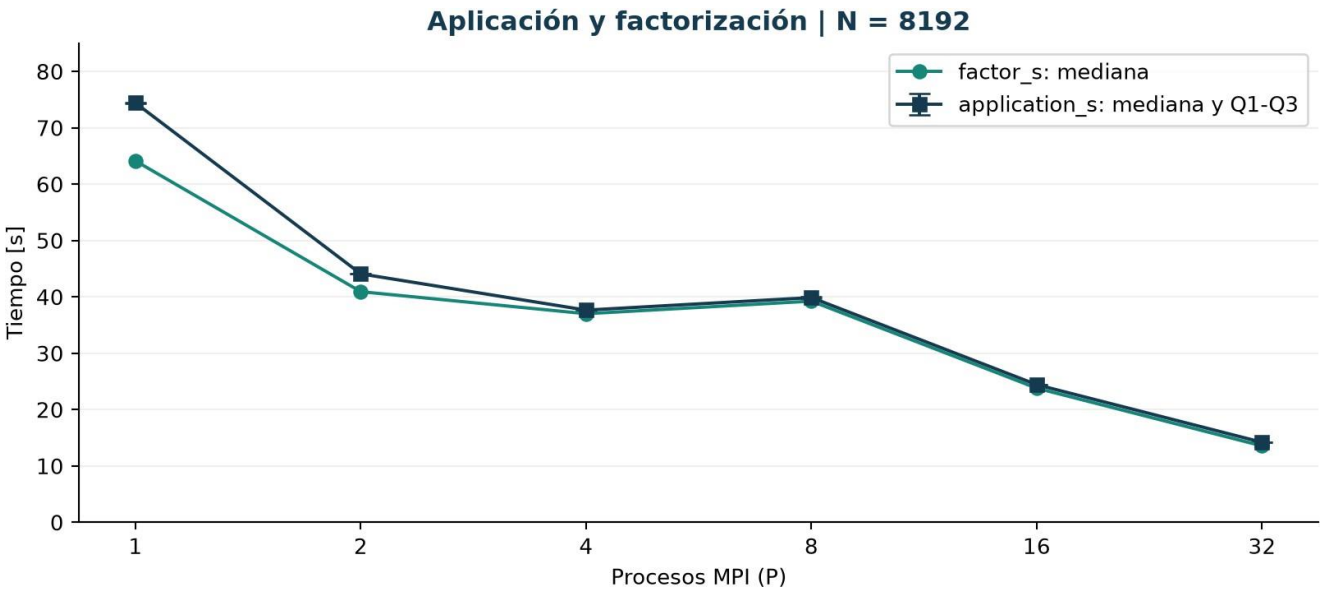


Figura 5. La aplicación incluye preparación y reconstrucción, pero excluye validación. Las barras de application_s son Q1-Q3 de siete observaciones.

P	Aplicación [s]	Q1 [s]	Q3 [s]	S aplicación
1	74.392908	74.349503	74.478477	1.000
2	44.068331	44.035932	44.084532	1.688
4	37.636445	37.602782	37.651906	1.977
8	39.863072	39.769258	39.956301	1.866
16	24.408448	24.404890	24.415617	3.048
32	14.185659	14.182349	14.192528	5.244

El tiempo mediano de aplicación pasa de 74.393 s a 14.186 s: speedup de 5.244×. Esta mejora difiere de la del kernel porque las etapas preparatorias tienen otro comportamiento con P.

La validación es trabajo adicional fuera de ambos tiempos. Por ello, estas métricas no deben usarse como una medida directa del tiempo total de una ejecución interactiva o de un trabajo de clúster.

Pilotos: evidencia exploratoria

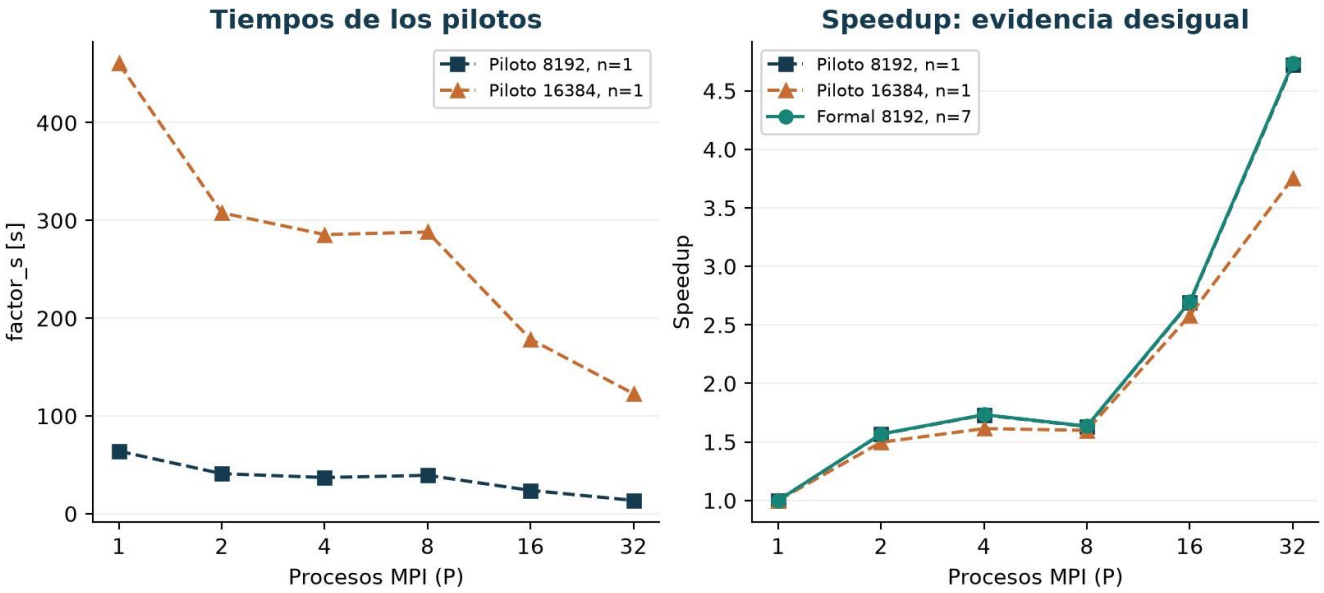


Figura 6. Cada piloto tiene una observación por P, sin barras de dispersión. La curva formal usa siete observaciones por P y se distingue explícitamente.

P	8192 factor [s]	16384 factor [s]	16384 aplic. [s]	S 16384	E 16384 [%]
1	63.998	460.668	501.806	1.000	100.00
2	40.887	307.788	328.366	1.497	74.84
4	36.984	285.529	292.584	1.613	40.33
8	39.219	288.170	290.724	1.599	19.98
16	23.794	178.518	180.976	2.581	16.13
32	13.552	122.774	125.420	3.752	11.73

Al duplicar N, el trabajo asintótico de Cholesky crece aproximadamente ocho veces y el almacenamiento denso cuatro veces. Eso no impone una razón de tiempos exacta: intervienen jerarquía de memoria, comunicaciones y fases medidas.

En N = 16384, P = 32 alcanza el menor tiempo observado: 122,77 s, aceleración de 3,752 respecto del piloto MPI con un proceso y eficiencia relativa del 11,73 %. P = 8 también tarda más que P = 4. Estos rasgos orientan una confirmación posterior, porque existe una sola observación por configuración.

El menor speedup frente a la serie formal N = 8192 no demuestra un deterioro estable al aumentar N. Ese contraste requiere repeticiones y condiciones comparables. Conservamos ambos pilotos como evidencia exploratoria, separados de la muestra formal.

Conclusiones

La campaña formal con siete observaciones por configuración demuestra que el programa MPI reduce su tiempo de factorización al pasar de uno a treinta y dos procesos: la mediana baja de 64,11 s a 13,54 s. La aceleración respecto del mismo MPI con P=1 alcanza 4,734 y la eficiencia relativa es 14,79 %. Treinta y dos procesos ofrecen la menor latencia ensayada, con rendimiento ampliamente sublineal respecto del ideal.

La curva presenta una degradación consistente dentro de esta sesión entre cuatro y ocho procesos. La mediana aumenta de 37,00 s a 39,23 s, aproximadamente 6,03 %, y las siete comparaciones por repetición tienen el mismo sentido. La separación de las observaciones respalda describir esa anomalía como repetida en la campaña. Su causa permanece abierta porque no se midieron componentes suficientes para atribuirla a colectivas, balance, memoria u otros mecanismos.

Al incluir preparación y reconstrucción, la mediana de aplicación disminuye de 74,39 s a 14,19 s, con aceleración de 5,244 respecto de su propia base P=1. La diferencia con la aceleración del kernel indica que las otras etapas responden de manera distinta al aumento de procesos. Ambos relojes excluyen validación y no representan el comando completo.

Alcance de la evidencia

La decisión depende del objetivo: P=32 minimiza el tiempo observado, mientras configuraciones menores conservan mayor eficiencia relativa. No se establece un óptimo global, comportamiento para P>32 ni escalabilidad multinodo. Esta campaña utiliza una cuota distinta de la campaña previa de seis CPUs equivalentes y debe conservarse como evidencia separada. La dispersión procede de una sesión y una matriz con semilla fija; no representa todas las entradas SPD.

LAPACK en el clúster

¿Cuánto margen de rendimiento local queda frente a una biblioteca numérica optimizada? Se comparan el kernel secuencial propio y LAPACKE_dpotrf_work, enlazado con OpenBLAS 0.3.34, dentro del mismo ejecutable y con la misma matriz. Es una comparación local de implementaciones; no es ScaLAPACK ni una prueba MPI de la biblioteca.

Qué mide cada campo

factor_s mide únicamente la llamada al kernel. total_s incluye reserva del arreglo y workspace, generación y factorización. Excluye preparar la matriz original/referencia del banco, validación y liberación. No existe aquí un reloj externo. Cada tamaño se ejecuta en un proceso con repeticiones internas, a diferencia de los lanzamientos nuevos de la campaña MPI.

Resultados: kernel y flujo completo del banco

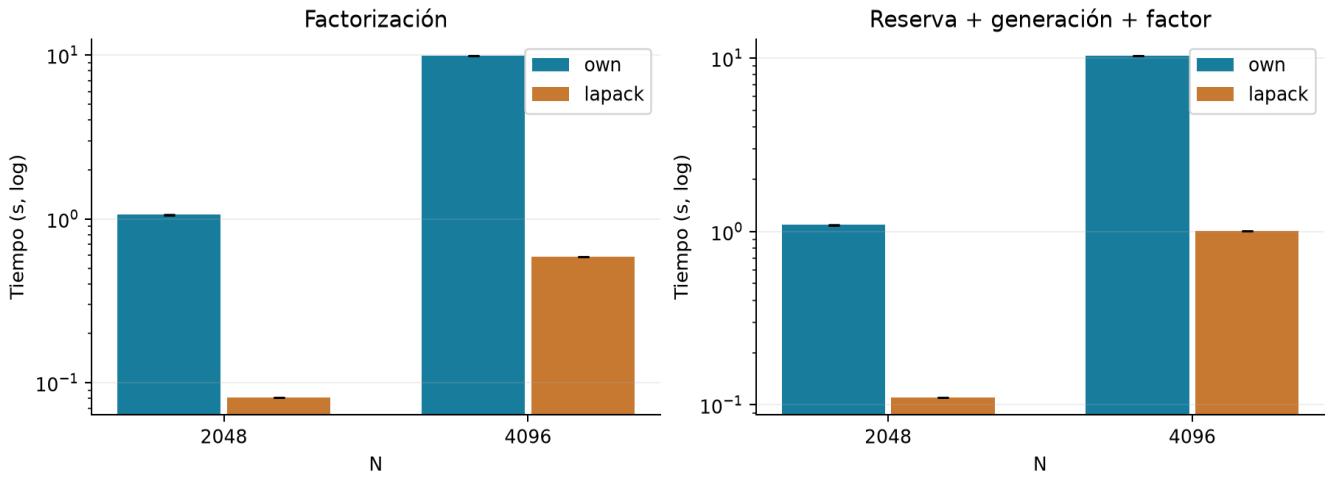


Figura 1. Medianas y Q1-Q3 inclusivos, seis muestras por método. Escala logarítmica. El total se resume directamente, sin sumar medianas de fases. La mejora es own/LAPACK dentro de este banco.

N	Reloj	Propio s	LAPACK s	Cociente
2048	Factor	1.060705	0.081292	13.05
2048	Total	1.089732	0.110383	9.87
4096	Factor	9.854616	0.589960	16.70
4096	Total	10.269754	1.006013	10.21

Qué responde la comparación

LAPACK/OpenBLAS con un hilo mantiene una ventaja amplia sobre el kernel secuencial propio en el entorno del clúster. Para N=2048, la mediana de factorización baja de 1,060705 s a 0,081292 s, con cociente de 13,05. Para N=4096, baja de 9,854616 s a 0,589960 s, con cociente de 16,70.

Al incluir reserva y generación en el total del banco, las mejoras son menores, pero siguen siendo importantes: 9,87 y 10,21, respectivamente. La diferencia entre los cocientes de kernel y total muestra que las fases compartidas limitan cuánto de la optimización de la factorización se conserva en el flujo medido.

El resultado confirma margen de mejora local en el candidato y justifica usar una biblioteca optimizada como referencia. La comparación no separa las técnicas internas que producen esa ventaja ni garantiza que aumentar procesos del kernel propio permita alcanzarla. Se refiere a dos métodos dentro del mismo banco secuencial y no incluye un reloj externo ni una implementación distribuida de LAPACK.

MPI frente a LAPACK en matrices grandes

Pregunta final: al crecer hasta N = 6144 y N = 8192, ¿MPI con seis procesos se acerca o supera a LAPACK/OpenBLAS con un hilo? La comparación corresponde a la campaña del 28/09/2026 y conserva las referencias, matrices y límites de reloj de cada banco.

Respuesta

MPI reduce el tiempo frente al C propio secuencial, pero no alcanza a LAPACK en ninguno de los dos tamaños. La razón MPI/LAPACK disminuye de 8,58 a 6,25: hay acercamiento relativo entre esos dos puntos, sin evidencia de un cruce futuro. La diferencia absoluta en segundos, en cambio, aumenta.

Qué hace comparable factor_s

El reloj principal abarca la fase que factoriza la misma familia de matrices SPD generadas con semilla 2026. En C/LAPACK es la llamada al kernel; en MPI es el intervalo del rango 0 alrededor del kernel distribuido, que incluye comunicaciones y esperas. No es un máximo reducido por fase ni una comparación de aritmética pura.

Generación y reunión quedan fuera de factor_s; los relojes secundarios tienen límites distintos. El banco C/LAPACK alterna métodos dentro de un proceso; MPI se lanza por separado. Por eso el cociente principal es descriptivo del costo de esa fase en los dos programas, no una prueba que aísle únicamente el efecto del paralelismo.

Resultados de factorización y dispersión

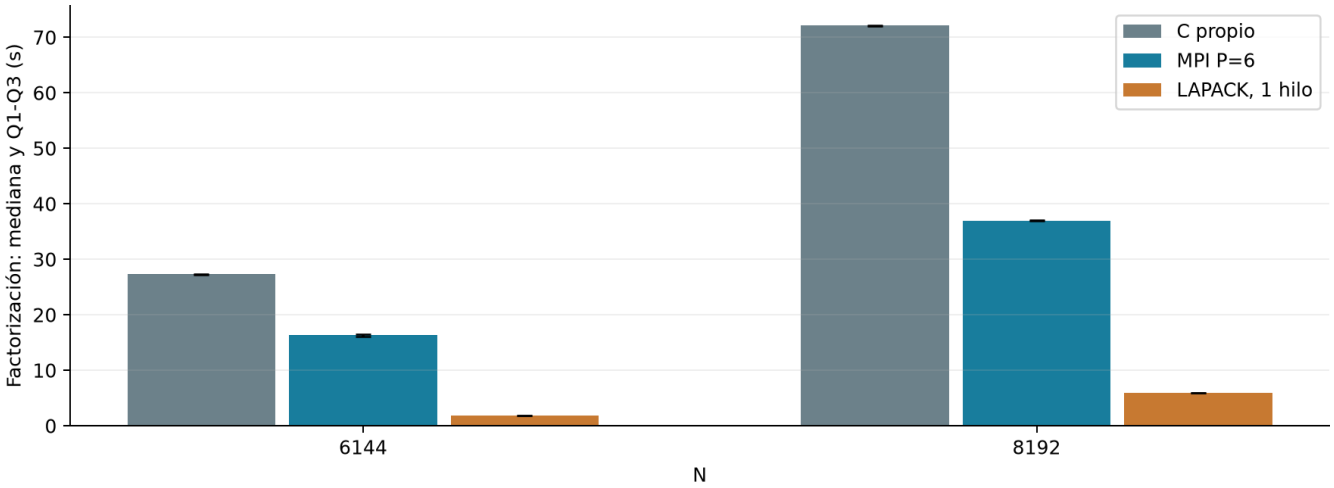


Figura 1. Medianas y Q1-Q3 inclusivos, seis observaciones útiles por grupo. Barras de cuartiles descriptivas; no intervalos de confianza. Los warmups no se incluyen.

N	Método	Q1 s	Mediana s	Q3 s	CV %
6144	own	27.2602	27.2690	27.2713	0.027
6144	mpi_p6	16.0909	16.2985	16.4927	1.424
6144	lapack	1.8987	1.8992	1.8996	0.076
8192	own	72.0020	72.0915	72.1250	0.131
8192	mpi_p6	36.9214	36.9539	36.9942	0.211
8192	lapack	5.9129	5.9159	5.9181	0.065

CV usa desviación muestral dividida por la media. La variación MPI en N=6144 es mayor que la de los otros grupos, pero la separación respecto de LAPACK es amplia: incluso el menor tiempo MPI supera al mayor LAPACK en ambos tamaños. Esta observación no necesita transformarse en un test de significación para describir la diferencia de esta sesión.

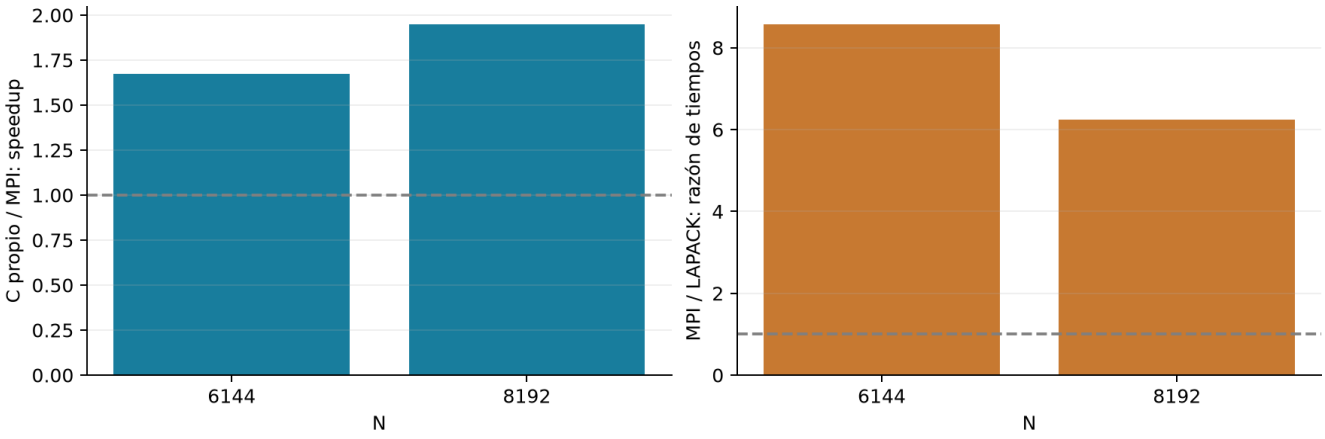


Figura 2. Izquierda: valores mayores que uno indican mejora MPI frente al C propio. Derecha: valores mayores que uno indican que MPI tarda más que LAPACK. Los denominadores responden preguntas distintas.

N	C / MPI	E MPI %	C / LAPACK	MPI / LAPACK
6144	1.673	27.9	14.359	8.582
8192	1.951	32.5	12.186	6.247

La eficiencia respecto del C propio es S/6: 27,9% y 32,5%. No equivale a utilización de CPU. LAPACK sigue por delante usando un hilo.

La diferencia absoluta MPI menos LAPACK pasa de 14.399 s a 31.038 s. Por tanto, 'se acerca' solo es correcto en la razón relativa entre estos tamaños. Dos puntos no permiten extrapolar un tamaño donde MPI alcance a la biblioteca ni establecer una tendencia monótona.

Interpretación y límite causal

Repartir el trabajo del kernel propio obtiene una mejora, pero no compensa la ventaja local de LAPACK/OpenBLAS en esta medición. La campaña no separa las contribuciones de localidad, vectorización, algoritmos internos de la biblioteca o comunicación. No hace falta identificar cada causa para responder la comparación experimental.

Relojes secundarios y condiciones del entorno

N	Método	Generación s	Reloj secundario	Mediana s
6144	own	0.4023	total_s	27.6713
6144	mpi_p6	0.1322	application_s	16.6069
6144	lapack	0.4023	total_s	2.3017
8192	own	3.9806	total_s	76.0697
8192	mpi_p6	0.0614	application_s	37.3368
8192	lapack	3.9786	total_s	9.8945

Estas medianas se presentan como contexto: no se construyen cocientes entre application_s y total_s. MPI application_s incluye reserva, generación, factorización y reunión/reconstrucción; total_s del banco own/LAPACK incluye reserva, generación y factor. Ninguno es el tiempo externo del comando. No se suman medianas para reconstruir totales.

Conclusión final de la comparación

Respuesta a la pregunta planteada

MPI con seis procesos acelera el kernel propio, pero LAPACK/OpenBLAS con un hilo conserva una ventaja amplia en los dos tamaños comparados. Para N=6144, las medianas de factorización son 27,2690 s para C propio, 16,2985 s para MPI y 1,8992 s para LAPACK. Para N=8192, son 72,0915 s, 36,9539 s y 5,9159 s, respectivamente. MPI obtiene aceleraciones de 1,67 y 1,95 frente al secuencial propio, mientras tarda 8,58 y 6,25 veces lo que tarda LAPACK.

El cociente MPI/LAPACK disminuye entre esos dos puntos, pero la diferencia absoluta aumenta de 14,399 s a 31,038 s. Hay un acercamiento relativo observado y, al mismo tiempo, una mayor diferencia en segundos. Dos tamaños no permiten extrapolar un cruce futuro. La separación entre observaciones de ambos métodos es amplia dentro de esta sesión y respalda la respuesta: MPI no alcanza a LAPACK en los casos ensayados.

La comparación principal describe el costo de la fase de factorización: en MPI incluye comunicaciones y esperas del intervalo registrado por rango 0. No aísla el efecto del paralelismo ni el aporte de cada optimización de la biblioteca. Los relojes secundarios tienen límites distintos y se conservan como contexto, sin construir cocientes entre application_s y total_s. La siguiente mejora debe estudiar también la eficiencia del kernel local, manteniendo una referencia especializada y condiciones de medición explícitas.

Conclusión general del proyecto

El proyecto muestra que una implementación correcta de Cholesky puede mejorar su rendimiento mediante cambios sucesivos en el acceso a datos, el reparto del trabajo y la preparación de la entrada, pero que cada mejora debe evaluarse con la referencia y el reloj adecuados. El vector contiguo transforma la competitividad del kernel right-looking; el vector local permite aprovechar mejor el reparto cíclico; y la generación distribuida reduce el costo de preparación conservando la misma matriz. Esas decisiones se apoyan en comparaciones medidas y en verificaciones de corrección.

MPI produce aceleraciones frente al código propio cuando el trabajo disponible compensa sus costos adicionales. La cantidad de procesos conveniente depende del tamaño, del entorno y del objetivo: reducir latencia o conservar eficiencia relativa. La ventaja del kernel o del flujo hasta L puede disminuir considerablemente al medir el comando completo. Las repeticiones de b=4 y las curvas no monótonas también muestran que una configuración favorable en una campaña necesita confirmación antes de adoptarse como regla.

La escalabilidad fuerte formal reduce la factorización de N=8192 de 64,11 s a 13,54 s con treinta y dos procesos, aunque con eficiencia relativa del 14,79 % y una degradación repetida de cuatro a ocho procesos. La comparación separada con seis procesos muestra, a su vez, que acelerar el kernel propio no basta para alcanzar LAPACK/OpenBLAS con un hilo en N=6144 y N=8192.

El aporte del trabajo es establecer qué decisiones mejoran estas implementaciones, en qué condiciones y con qué límites. La evidencia obtenida orienta una continuación centrada en elevar el rendimiento local, reducir costos del flujo y evaluar paralelismo distribuido con mediciones más específicas. Las conclusiones corresponden a las matrices, campañas y recursos documentados; la ejecución multinodo y las variantes por bloques o híbridas permanecen como extensiones por comprobar.

Continuación técnica razonable

La primera continuación es estudiar un Cholesky realmente bloqueado con operaciones BLAS de nivel 3. La ventaja de LAPACK respalda esa dirección como hipótesis de mejora del kernel local, aunque las campañas actuales no cuantifican qué fracción de la diferencia podría recuperar una nueva versión. La evaluación debería conservar la corrección y comparar fases equivalentes contra el código actual y la biblioteca.

OpenMP y una versión híbrida MPI con hilos permitirían estudiar otras formas de paralelismo dentro de cada nodo. Esa comparación requiere mantener el mismo presupuesto de recursos, controlar afinidad y coordinar la cantidad de hilos con BLAS si se usa una biblioteca multihilo. Su conveniencia permanece por medir.

Para explicar y reducir las limitaciones observadas, las siguientes mediciones deberían registrar tiempos por rango y componentes de cálculo, colectivas y espera; revisar la validación centralizada y la reconstrucción; y contrastar los comportamientos no monótonos en nuevas sesiones. Una campaña multinodo y una referencia como ScaLAPACK ampliarían la evaluación a distribuciones y costos de comunicación que las pruebas actuales no cubren. Estas extensiones se proponen por las preguntas que deja la evidencia, sin anticipar ganancias todavía no medidas.